Branch-and-Price Strikes Back for the k-Vertex Cut Problem

Fabio Ciccarelli, Fabio Furini, Christopher Hojny, and Marco Lübbecke

Sapienza University of Rome; Eindhoven University of Technology; RWTH Aachen University

Abstract

Given an undirected graph, the k-vertex cut problem asks for a minimum-cost set of vertices whose removal yields at least k connected components. We present an extended integer linear programming formulation that unifies and strengthens existing models, and a branch-and-price algorithm with tailored branching rules, valid inequalities, and symmetry handling. The formulation dominates previous models in terms of linear relaxations. Computational experiments show substantial improvements over state-of-the-art methods in both optimal solutions and running times.

Keywords: graph partitioning; critical vertex detection problems; integer linear programming; branch-and-price.

1 Introduction

Graph disconnection problems via critical vertex deletion are fundamental in network optimization, due to their theoretical significance as well as their broad applications in telecommunication and transportation systems, computational biology, network vulnerability assessment, and immunization against epidemics (Lalou et al. 2018). As connectivity underlies the behavior, function, and resilience of complex networks, understanding the mechanisms through which it can be altered, influenced, or reorganized is important for both graph theory and real-world applications. Within this broad family of *critical vertex detection problems* (CVDPs), a prominent class is concerned with fragmenting the graph through the deletion of vertices.

In this paper, we focus on the *k-vertex cut problem* (k-VCP), an important CVDP that has received considerable attention in recent years; see, e.g., Marx (2006), Berger et al. (2014), Cornaz et al. (2019), Furini et al. (2020).

Formally, let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ be a simple undirected graph, where $\mathcal{V}$ denotes the set of vertices and $\mathcal{E}$ the set of edges. For any subset of vertices $S \subseteq \mathcal{V}$, the induced subgraph $\mathcal{G}[S] = (S, \mathcal{E}[S])$ consists of the vertices in S together with the edges

$$\mathcal{E}[S] = \{\, \{u, v\} \in \mathcal{E} : u, v \in S \,\},$$

i.e., all edges of $\mathcal{G}$ whose endpoints lie in S. A *connected component* is an inclusion-wise maximal connected induced subgraph of $\mathcal{G}$. A *vertex cut* is a subset of vertices $\mathcal{V}_0 \subseteq \mathcal{V}$ whose removal disconnects the graph, i.e., the induced subgraph $\mathcal{G}[\mathcal{V} \setminus \mathcal{V}_0]$ contains at least two connected components. If the induced subgraph contains at least k connected components, then $\mathcal{V}_0$ is called a *k-vertex cut*. The (weighted) k-VCP can therefore be formally defined as follows:

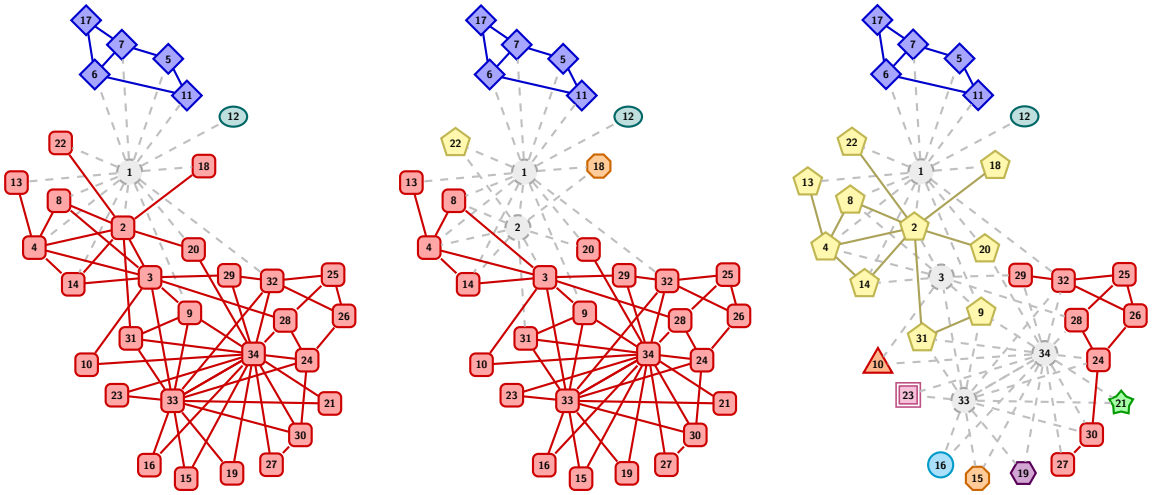
Definition 1 (k-vertex cut problem)**.** *Given a graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, a non-negative cost c_v for each vertex $v \in \mathcal{V}$, and an integer $k \geq 2$, the* k-vertex cut problem *(k-VCP) consists in finding a minimum-cost set $\mathcal{V}_0 \subseteq \mathcal{V}$ such that the induced subgraph $\mathcal{G}[\mathcal{V} \setminus \mathcal{V}_0]$ contains at least k connected components.*

An equivalent structural characterization of k-vertex cuts that we use throughout the paper is as follows. A subset of vertices $\mathcal{V}_0 \subseteq \mathcal{V}$ is a k-vertex cut if and only if the induced subgraph $\mathcal{G}[\mathcal{V} \setminus \mathcal{V}_0]$ admits a partition of its vertex set into k non-empty subsets $\mathcal{V}_1, \mathcal{V}_2, \ldots, \mathcal{V}_k$ that are pairwise disconnected, in the sense that no edge of $\mathcal{E}$ has endpoints in two different subsets. Each subset may contain one or more connected components; what matters is that the k subsets are mutually disconnected in the induced subgraph $\mathcal{G}[\mathcal{V} \setminus \mathcal{V}_0]$. For ease of reference, in the remainder of the paper, we will refer to each subset $\mathcal{V}_1, \mathcal{V}_2, \ldots, \mathcal{V}_k$ as a *cluster* of the graph. From a complementary perspective, the k-VCP can also be viewed as finding a maximum-cost set of vertices $\bigcup_{i=1}^{k} \mathcal{V}_i$ such that the induced subgraph $\mathcal{G}\left[\bigcup_{i=1}^{k} \mathcal{V}_i\right]$ contains at least k connected components.

Figure 1 illustrates optimal solutions of the k-VCP on the classical Zachary's karate club network (Zachary 1977) for $k \in \{3, 5, 10\}$ with unit vertex costs. The vertices of the graph are labeled with the numbers displayed inside each node. This instance has also been used to illustrate optimal solutions of other related optimization problems on graphs; see, e.g., Furini et al. (2019, 2021, 2022). For each value of k, the corresponding subfigure shows the vertices of an optimal k-vertex cut in gray, together with their incident edges drawn as dashed lines to indicate the portion of the graph removed by the vertex cut. The remaining vertices are partitioned into exactly k connected components, each highlighted by a different color and shape. For $k = 3$, it is sufficient to remove a single vertex; for $k = 5$, two vertices must be removed; and for $k = 10$, four vertices are required. The figure illustrates both the increasing fragmentation of the graph and the change in the structure of optimal k-vertex cuts as k grows. Further details regarding these optimal solutions are provided in Appendix B.

As for the specific applications of the k-VCP, this problem plays a central role whenever the decision maker aims to disrupt communication, isolate subnetworks, or assess the robustness and structural vulnerability of complex systems; see e.g., Furini et al. (2020). The computational complexity of the k-VCP depends critically on the value of k. When the deletion budget is taken as parameter, the decision version of the problem is W[1]-hard (Marx 2006). If k is part of the input, the problem is already $\mathcal{NP}$-hard, as follows from the hardness of the k-way vertex cut problem (Berger et al. 2014), whose decision version coincides with the decision version of the k-VCP. Moreover, even when k is fixed, the problem remains $\mathcal{NP}$-hard for every $k \geq 3$ (Cornaz et al. 2019). On the other hand, the case $k = 2$ is polynomially solvable, as it reduces to computing $O(|\mathcal{V}|^2)$ minimum s–t cuts via vertex-splitting over all pairs of vertices, which in the unit-cost case coincides with computing the vertex connectivity of the graph.

Three main approaches currently represent the most effective exact methods for solving the k-VCP. A first, off-the-shelf option consists in using a general-purpose *integer linear programming* (ILP) solver applied to a compact ILP formulation with a polynomial number of variables and constraints, as proposed in Cornaz et al. (2019). Beyond this generic strategy, two dedicated exact algorithms have been introduced in the literature: a *branch-and-price* (BP)



(a) $k = 3$ (b) $k = 5$ (c) $k = 10$

Figure 1: Optimal k-vertex cuts (gray vertices with dashed incident edges) for Zachary's karate club network with unit vertex costs, shown for $k \in \{3, 5, 10\}$. The colored vertices represent the connected components remaining in the graph.

algorithm (Cornaz et al. 2019), based on an ILP formulation with an exponential number of variables, and a *branch-and-cut* (BC) algorithm (Furini et al. 2020), based on an ILP formulation with an exponential number of constraints. Although these exact methods are quite effective, they still fail to solve many benchmark instances to optimality. Motivated by these limitations, the main methodological contribution of this paper is to introduce a new exact algorithm for the k-VCP that markedly improves upon the BP approach and enables the solution of significantly larger and more challenging instances.

1.1 ILP models for the k-VCP from the literature

In this section we recall the three ILP formulations that have been proposed in the literature for the k-VCP. For each model, we outline its modeling structure and the exact solution method originally associated with it. These models, together with their corresponding algorithms, will be included in the computational experiments of this article and used as benchmarks against the new BP algorithm proposed in this paper.

1.1.1 The compact ILP model

Let $\mathcal{K}$ denote the set of cluster indices, namely $\{1, 2, \ldots, k\}$. For each vertex $v \in \mathcal{V}$ and each index $i \in \mathcal{K}$, let y_{iv} be a binary variable taking value 1 if v is assigned to the i-th cluster of the graph and value 0 otherwise. Vertices assigned to at least one cluster are retained in the graph, whereas vertices not assigned to any cluster belong to the set $\mathcal{V}_0$ and form the k-vertex cut. Using these variables, the *compact* ILP formulation for the k-VCP, introduced by Cornaz

et al. (2019) and denoted by ILP_C, reads as follows:

$$(\text{ILP}_\text{C}) \qquad \max_{\boldsymbol{y}\in\{0,1\}^{\mathcal{K}\times\mathcal{V}}} \sum_{i\in\mathcal{K}}\sum_{v\in\mathcal{V}} c_v\, y_{iv} \tag{1a}$$

$$\sum_{i\in\mathcal{K}} y_{iv} \le 1, \qquad v\in\mathcal{V}, \tag{1b}$$

$$y_{iu} + \sum_{j\in\mathcal{K}\setminus\{i\}} y_{jv} \le 1, \qquad i\in\mathcal{K},\ \{u,v\}\in\mathcal{E}, \tag{1c}$$

$$\sum_{v\in\mathcal{V}} y_{iv} \ge 1, \qquad i\in\mathcal{K}. \tag{1d}$$

This formulation is closely related to the one by Borndörfer et al. (1998) for the k-separator problem and uses the same assignment variables. The objective function (1a) maximizes the total cost of vertices assigned to the clusters of the graph. Throughout the paper, bold symbols below the optimization operator indicate the decision variables of the model together with their domains. Constraints (1b) ensure that each vertex is assigned to at most one cluster. Constraints (1c) prevent adjacent vertices from being assigned to different clusters, thereby ensuring mutual disconnection of the clusters. Constraints (1d) require each cluster to be non-empty, enforcing the presence of at least k connected components. The compact formulation involves a polynomial number of binary variables and constraints, and can therefore be solved directly by a general-purpose ILP solver.

1.1.2 The extended ILP model

Let $\mathscr{S}\subseteq 2^\mathcal{V}$ be the family of all non-empty subsets of vertices, where $2^\mathcal{V}$ denotes the power set of $\mathcal{V}$. Hence, $\mathscr{S} = \{S\subseteq\mathcal{V} : S\neq\emptyset\}$. Furthermore, for any $U\subseteq\mathcal{V}$, let $\mathscr{S}(U) = \{S\in\mathscr{S} : S\cap U\neq\emptyset\}$ denote the subsets in $\mathscr{S}$ that contain at least one vertex of U. For singleton sets $\{v\}$, $v\in\mathcal{V}$, we write $\mathscr{S}(v)$ instead of $\mathscr{S}(\{v\})$, i.e., $\mathscr{S}(v) = \{S\in\mathscr{S} : v\in S\}$ denotes the subsets in $\mathscr{S}$ that contain vertex v. For each subset $S\in\mathscr{S}$, let λ_S be a binary variable taking value 1 if S is selected as one of the k clusters of the solution and value 0 otherwise. Denote by $\mathscr{C}$ the family of all cliques in $\mathcal{G}$. Let $\mathscr{C}'\subseteq\mathscr{C}$ be an *edge-covering* family of cliques of $\mathcal{G}$, that is, a family such that for every edge $\{u,v\}\in\mathcal{E}$ there exists at least one clique $C\in\mathscr{C}'$ containing both u and v. Using the $\boldsymbol{\lambda}$-variables, the *extended* ILP formulation for the k-VCP, introduced by Cornaz et al. (2019) and denoted by ILP_E1, reads as follows:

$$(\text{ILP}_\text{E1}) \qquad \max_{\boldsymbol{\lambda}\in\{0,1\}^\mathscr{S}} \sum_{S\in\mathscr{S}} \left(\sum_{v\in S} c_v\right) \lambda_S \tag{2a}$$

$$\sum_{S\in\mathscr{S}} \lambda_S = k, \tag{2b}$$

$$\sum_{S\in\mathscr{S}(v)} \lambda_S \le 1, \qquad v\in\mathcal{V}, \tag{2c}$$

$$\sum_{S\in\mathscr{S}(C)} \lambda_S \le 1, \qquad C\in\mathscr{C}', \tag{2d}$$

The objective function (2a) maximizes the total cost of the k clusters selected in the solution. It generalizes the original unweighted version—which maximizes the cardinality $|S|$ of each

chosen subset—by incorporating vertex costs. Constraint (2b) ensures that exactly k subsets are selected, and thus that the solution consists of k clusters. Constraints (2c) require that each vertex $v \in \mathcal{V}$ appears in at most one selected subset, guaranteeing that the chosen subsets are vertex-disjoint and therefore represent distinct clusters. Constraints (2d) enforce that the selected clusters are mutually disconnected. Since $\mathscr{C}'$ is an edge-covering family of cliques, requiring that at most one subset in $\mathscr{S}(C)$ can be selected for each $C \in \mathscr{C}'$ ensures that no two chosen subsets contain adjacent vertices. Consequently, the k selected clusters of vertices are pairwise disconnected in the graph. Figure 2 illustrates the logic of Constraints (2d) using a small graph with six vertices and an edge-covering family of cliques given by its edges. As an example, consider the clique $C = \{b, e\}$ and two potential clusters $S_1 = \{a, b, c\}$ and $S_2 = \{d, e, f\}$. Since $S_1 \cap C = \{b\}$ and $S_2 \cap C = \{e\}$, both clusters belong to $\mathscr{S}(C)$. Therefore, Constraint (2d) associated with C prevents the variables corresponding to S_1 and S_2 from both taking value 1. Indeed, selecting both clusters would violate the requirement that the selected clusters be pairwise disconnected. In Cornaz et al. (2019), a family of cliques of polynomial size with respect to the number of edges of the graph is considered.

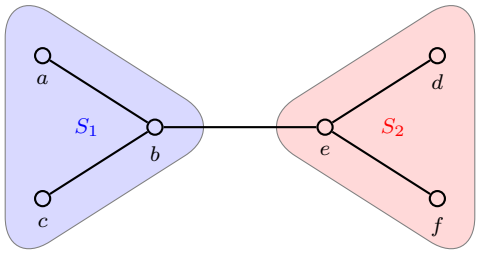


Figure 2: Illustration of Constraint (2d). The candidate clusters $S_1 = \{a, b, c\}$ (blue) and $S_2 = \{d, e, f\}$ (red) both intersect the clique $C = \{b, e\}$, so Constraint (2d) forbids selecting them simultaneously.

The extended formulation ILP_{E1} contains an exponential number of variables. As a consequence, solving the model requires a *column generation* (CG) approach embedded within a BP algorithm. At each node of the branching tree, the LP relaxation of Formulation (2) is solved through a CG procedure, where new variables are generated by solving the so-called *pricing problem* (PP). We refer the interested reader to Barnhart et al. (1998), Lübbecke & Desrosiers (2005) for further details on CG and BP algorithms, as well as to the book by Desaulniers et al. (2005) and the two recent monographs by Desrosiers et al. (2026) and Uchoa et al. (2024). In what follows, we briefly describe the two principal components of the BP algorithm of Cornaz et al. (2019): the pricing problem and the branching scheme.

At the root node of the BP algorithm, Cornaz et al. (2019) show that the pricing problem can be solved via solving a sequence of minimum cut problems on an auxiliary network, enabling the use of efficient max-flow/min-cut algorithms. Although the pricing problem is originally described for unit vertex costs, the same approach extends naturally to the weighted setting by simply adjusting the vertex profits in the reduced-cost computation, and both the complexity and the solution methods remain unchanged. We refer the reader to Section 3.1.2 for further details.

A dedicated branching scheme is proposed in Cornaz et al. (2019), operating on two levels. At the first level, a "partially" assigned vertex $v \in \mathcal{V}$ is identified, namely one for which the sum of the variables over subsets containing v is fractional. Two branches are created: one forcing v into the vertex cut and the other forcing v to belong to exactly one cluster. This branching preserves the structure of the pricing problem: in the first branch, v is simply forbidden in newly generated subsets, while in the second the pricing formulation is unchanged. Since this is not sufficient to guarantee integrality, a second branching level, akin to Ryan-Foster branching (Ryan & Foster 1981), is applied to vertex pairs. Here, two vertices $u \in \mathcal{V}$ and $v \in \mathcal{V}$ outside the vertex cut that are "partially" assigned to the same cluster (i.e., the sum of variables over subsets containing both is fractional) trigger two additional branches: enforcing that they belong to the same cluster or to different clusters. In the "same-cluster" branch, the pricing problem is preserved by contracting $\{u, v\}$ into a single supervertex, whereas in the "different-clusters" branch an incompatibility constraint forbidding subsets containing both vertices must be enforced. The latter destroys the min-cut structure of the pricing problem, which must then be solved using an ILP formulation.

1.1.3 The natural ILP model

A *subgraph* of $\mathcal{G}$ is any graph $\mathcal{F} = (\mathcal{V}(\mathcal{F}), \mathcal{E}(\mathcal{F}))$ such that $\mathcal{V}(\mathcal{F}) \subseteq \mathcal{V}$ and $\mathcal{E}(\mathcal{F}) \subseteq \mathcal{E}$, with every edge of $\mathcal{E}(\mathcal{F})$ having both endpoints in $\mathcal{V}(\mathcal{F})$. For any subgraph $\mathcal{F}$ of $\mathcal{G}$, we denote by $\deg_{\mathcal{F}}(v)$ the degree of vertex v in $\mathcal{F}$, i.e., the number of edges of $\mathcal{F}$ incident to v. A subgraph $\mathcal{F}$ is *spanning* if $\mathcal{V}(\mathcal{F}) = \mathcal{V}$, and it is *cycle-free* if it contains no cycles (equivalently, if $\mathcal{F}$ is a *forest*). We denote by $\mathscr{F}_{\mathcal{G}}$ the family of all *spanning cycle-free subgraphs* of $\mathcal{G}$, that is,

$$\mathscr{F}_{\mathcal{G}} = \{\, \mathcal{F} = (\mathcal{V}, \mathcal{E}(\mathcal{F})) : \mathcal{F} \text{ is acyclic} \,\}.$$

For each vertex $v \in \mathcal{V}$, we introduce a binary variable x_v that takes value 1 if v belongs to the k-vertex cut and 0 otherwise. Using the $\boldsymbol{x}$-variables, the *natural* ILP formulation for the k-VCP, introduced by Furini et al. (2020) and denoted by $\mathrm{ILP_N}$, reads as follows:

$$(\mathrm{ILP_N}) \qquad \min_{\boldsymbol{x} \in \{0,1\}^{\mathcal{V}}} \sum_{v \in \mathcal{V}} c_v \, x_v \tag{3a}$$

$$\sum_{v \in \mathcal{V}} \big(\deg_{\mathcal{F}}(v) - 1 \big) x_v \geq k - |\mathcal{V}| + |\mathcal{E}(\mathcal{F})|, \qquad \mathcal{F} \in \mathscr{F}_{\mathcal{G}}. \tag{3b}$$

The objective function (3a) minimizes the total weight of the vertices removed in the k-vertex cut. Constraints (3b) form an exponential family of inequalities derived from a bilevel interpretation of the k-VCP and from the key property that a graph has at least k connected components if and only if every acyclic spanning subgraph contains at most $|\mathcal{V}| - k$ edges. We refer to Furini et al. (2020) for further details.

The natural formulation $\mathrm{ILP_N}$ contains an exponential number of constraints. As a consequence, solving the model requires a *cut generation* approach embedded within a branch-and-cut algorithm. At each node of the branching tree, the LP relaxation of Formulation (3) is solved by iteratively generating violated inequalities from the family (3b). These cuts are produced by solving a separation problem that searches for a maximum-weight acyclic subgraph of the current residual graph. As shown by Furini et al. (2020), this separation can be performed in polynomial time for both integer and fractional solutions, allowing the exponential family of

constraints to be added only when violated.

1.2 Paper outline and contributions

This section reviews the main contributions of our work and specifies the sections in which they are presented. Section 2 presents our main theoretical contribution: a new extended ILP formulation that unifies and extends both the natural formulation and the previous extended formulation by simultaneously making use of the binary variables from both models. Crucially, we prove that integrality needs to be enforced only on the variables inherited from the natural formulation, whereas the variables coming from the extended formulation can be safely relaxed to continuous without affecting optimality—an essential property that greatly simplifies the design of the new branch-and-price algorithm. The *linear programming* (LP) relaxation of the new extended model strictly dominates the LP relaxations of both classical models, thereby providing a substantially tighter foundation for exact solution methods. Building on the new formulation, Section 3 introduces our main methodological contributions, namely a branch-and-price algorithm that incorporates a tailored column generation procedure, a branching strategy fully compatible with the pricing problem, and dedicated symmetry-handling and primal-heuristic components that improve the computational performance. Section 4 reports computational results on standard benchmark instances from the literature, including comparisons with state-of-the-art exact methods and ablation tests designed to assess the contribution of each main component of our new approach. The experiments show that our method establishes a new state-of-the-art for the k-VCP, outperforming all existing exact algorithms and solving 73 open instances within the time limit of one hour. Finally, Section 5 summarizes the contributions of the paper and outlines directions for future research.

2 A new extended ILP model for the k-VCP

In this section, we introduce a new extended ILP formulation for the k-VCP that simultaneously exploits the vertex-cut variables $\boldsymbol{x} \in \{0,1\}^{\mathcal{V}}$ of ILP_{N} and the cluster-selection variables $\boldsymbol{\lambda} \in \{0,1\}^{\mathscr{S}}$ of ILP_{E1}. By combining the strengths of both formulations within a unified model, we obtain an enhanced representation of the k-VCP given by ILP_{E2} below. We will show, through a detailed theoretical study, that the variables inherited from ILP_{E1} can in fact be treated as continuous. The new model therefore does not only shed light on hidden properties of ILP_{E1}, but has also important algorithmic consequences for solving the k-VCP as we will discuss in detail in Section 3. As in ILP_{N}, variable x_v indicates whether vertex $v \in \mathcal{V}$ is contained in the vertex cut, and, analogously to ILP_{E1}, variable λ_S indicates whether $S \in \mathscr{S}$ forms one of the k mutually disconnected clusters of the remaining graph. Using these variables, the new *extended*

ILP formulation for the k-VCP and denoted by ILP_{E2}, reads as follows:

$$(\text{ILP}_{\text{E2}}) \qquad \min_{\substack{\boldsymbol{x}\in\{0,1\}^{\mathcal{V}}\\ \boldsymbol{\lambda}\in\{0,1\}^{\mathscr{S}}}} \sum_{v\in\mathcal{V}} c_v\, x_v \tag{4a}$$

$$\sum_{S\in\mathscr{S}} \lambda_S = k, \tag{4b}$$

$$\sum_{S\in\mathscr{S}(v)} \lambda_S + x_v = 1, \qquad v \in \mathcal{V}, \tag{4c}$$

$$\sum_{S\in\mathscr{S}(C)} \lambda_S \leq 1, \qquad C \in \mathscr{C}. \tag{4d}$$

The objective function (4a) coincides with the one of ILP_{N} and minimizes the total cost of the the selected k-vertex cut. Constraints (4b) and (4d) serve the same purpose as in ILP_{E1}, i.e., they guarantee to select exactly k mutually disconnected clusters and enforce adjacent vertices in the remaining graph to be contained in the same cluster, respectively. Nonetheless, unlike the formulation of Cornaz et al. (2019), where the corresponding constraints are imposed on an edge-covering family of cliques, Constraints (4d) are defined over the whole clique family $\mathscr{C}$. This yields a stronger formulation, but also introduces an exponential number of inequalities. Therefore, these constraints require a tailored separation procedure, described in Section 3.1.4. Furthermore, it is worth noticing that it is sufficient to define Constraints (4d) only over the family of maximal cliques of $\mathcal{G}$. Indeed, if $C_1 \subseteq C_2$ for two cliques $C_1, C_2 \in \mathscr{C}$, then $\mathscr{S}(C_1) \subseteq \mathscr{S}(C_2)$, and therefore the constraint associated with C_1 is dominated by the one associated with C_2. To explain Constraints (4c), observe that (2c) in ILP_{E1} takes left-hand side value 1 if $v \in \mathcal{V}$ is not removed from the graph and of 0 otherwise. That is, x_v in (4c) can be interpreted as a slack variable in Constraint (2c). The correctness of ILP_{E2} immediately follows from the correctness of ILP_{E1}.

We now discuss the main theoretical contribution of this section. Notice the domain of the $\boldsymbol{\lambda}$-variables.

Theorem 1. *Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ be an undirected graph, let $\boldsymbol{c} \in \mathbb{R}^{\mathcal{V}}$, and let k be a positive integer. Then,*

$$\min_{\substack{\boldsymbol{x}\in\{0,1\}^{\mathcal{V}}\\ \boldsymbol{\lambda}\geq\boldsymbol{0}}} \left\{ \sum_{v\in\mathcal{V}} c_v x_v : \text{(4b)}, \text{(4c)}, \textit{ and } \text{(4d)} \right\} \tag{5}$$

is a correct model of the k-vertex cut problem. That is, for every optimal solution $(\boldsymbol{x}, \boldsymbol{\lambda})$ of (5), the set of vertices $\{v \in \mathcal{V} : x_v = 1\}$ is an optimal solution of the k-vertex cut problem.

In the remainder of this section, we prove Theorem 1 and compare the strength of the LP relaxations of ILP_{N}, ILP_{E1}, and ILP_{E2}.

2.1 Partially relaxing integrality in ILP_{E2}

To prove Theorem 1, we proceed in two steps. First, we derive an alternative formulation of the k-VCP that contains both the $\boldsymbol{x}$-variables and $\boldsymbol{\lambda}$-variables as well as additional variables. An important feature of this model is that integrality is only assumed for the $\boldsymbol{x}$-variables.

Afterwards, we show that, although only the $\boldsymbol{x}$-variables are enforced to take integer values, there always exists an optimal solution in which all variables take integer values. This fact is exploited by adding further cutting planes and projecting this model to (5).

2.1.1 An alternative formulation

To derive our alternative formulation, we make use of an observation by Furini et al. (2020). Let $\boldsymbol{x} \in \{0,1\}^{\mathcal{V}}$ and let $\tilde{\mathcal{V}} = \mathcal{V} \setminus \{v \in \mathcal{V} : x_v = 1\}$. We define the reduced graph $\mathcal{G}(\boldsymbol{x}) = (\tilde{\mathcal{V}}, \mathcal{E}[\tilde{\mathcal{V}}]) = (\tilde{\mathcal{V}}, \tilde{\mathcal{E}})$. That is, $\mathcal{G}(\boldsymbol{x})$ is the induced subgraph of $\mathcal{G}$ that arises by removing the vertices $v \in \mathcal{V}$ with $x_v = 1$. Let $\Phi(\boldsymbol{x})$ denote the maximum number of edges in a spanning forest of $\mathcal{G}(\boldsymbol{x})$. Then, Furini et al. (2020) have shown that the k-VCP can be modeled as

$$\min_{\boldsymbol{x}\in\{0,1\}^{\mathcal{V}}} \left\{ \sum_{v\in\mathcal{V}} c_v x_v : \Phi(\boldsymbol{x}) \leq n - k - \sum_{v\in\mathcal{V}} x_v \right\}. \tag{6}$$

Since (6) involves the non-linear constraint $\Phi(\boldsymbol{x}) \leq n-k-\sum_{v\in\mathcal{V}} x_v$, Model (6) cannot be solved by standard ILP solvers. Furini et al. (2020) therefore linearize this constraint using cutting planes. For deriving our alternative model, we instead express $\Phi(\boldsymbol{x})$ as an LP model.

Lemma 2. *Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ be an undirected graph and let $\boldsymbol{x} \in \{0,1\}^{\mathcal{V}}$. Then,*

$$\Phi(\boldsymbol{x}) = \max_{\boldsymbol{y}\geq\boldsymbol{0}} \sum_{e\in\mathcal{E}} y_e$$

$$y(S) \leq |S| - 1, \qquad S \subseteq \mathcal{V},\ |S| \geq 2, \qquad (\lambda_S)$$

$$y_e \leq 1 - x_u, \qquad e = \{u,v\} \in \mathcal{E}, \qquad (\beta_{eu})$$

$$y_e \leq 1 - x_v, \qquad e = \{u,v\} \in \mathcal{E}, \qquad (\beta_{ev})$$

where $y(S)$ denotes $\sum_{e\in\mathcal{E}[S]} y_v$.

Proof. Proof This is a consequence of a result by Edmonds (1970) that yields an inequality description of the convex hull of incidence vectors of acyclic subgraphs, see Appendix A for details. □ □

Based on Lemma 2, we know that $\Phi(\boldsymbol{x})$ can be computed by solving an LP model. That is, Model (6) is a bilevel optimization problem with an LP model in the lower level. We can turn the problem into a single-level problem by dualizing the LP model for computing $\Phi(\boldsymbol{x})$. To derive the dual, we use the dual variables as indicated by the Greek letters in brackets in the LP in Lemma 2, and introduce the notation $\mathscr{S}^{\subseteq}(U) \coloneqq \{S \in \mathscr{S} : U \subseteq S\}$ for $U \subseteq \mathcal{V}$, denoting the family of all supersets of U in $\mathscr{S}$. The dual then reads as

$$\Phi(\boldsymbol{x}) = \min_{\substack{\boldsymbol{\lambda}\geq\boldsymbol{0}\\ \boldsymbol{\beta}\geq\boldsymbol{0}}} \sum_{\substack{S\in\mathscr{S}:\\ |S|\geq 2}} (|S|-1)\lambda_S + \sum_{e=\{u,v\}\in\mathcal{E}} ((1-x_u)\beta_{eu} + (1-x_v)\beta_{ev}) \tag{7a}$$

$$\sum_{S\in\mathscr{S}^{\subseteq}(e)} \lambda_S + \beta_{eu} + \beta_{ev} \geq 1, \quad e = \{u,v\} \in \mathcal{E}. \tag{7b}$$

For notational convenience, in the following we also include variables λ_S for $S \in \mathscr{S}$ with $|S| = 1$. This is possible without altering the structure of the problem, because these variables have neither a contribution to the objective function nor Constraint (7b).

Based on the dual, we can substitute $\Phi(\boldsymbol{x})$ in (6) and obtain

$$\min_{\substack{\boldsymbol{x}\in\{0,1\}^{\mathcal{V}} \\ \boldsymbol{\lambda}\geq\mathbf{0} \\ \boldsymbol{\beta}\geq\mathbf{0}}} \sum_{v\in\mathcal{V}} c_v x_v \tag{8a}$$

$$\sum_{S\in\mathscr{S}} (|S|-1)\lambda_S + \sum_{e=\{u,v\}\in\mathcal{E}} ((1-x_u)\beta_{eu} + (1-x_v)\beta_{ev}) \leq n-k-\sum_{v\in\mathcal{V}} x_v, \tag{8b}$$

$$\sum_{S\in\mathscr{S}^{\subseteq}(e)} \lambda_S + \beta_{eu} + \beta_{ev} \geq 1, \qquad e=\{u,v\}\in\mathcal{E}. \tag{8c}$$

This single-level reformulation forms the basis for our alternative model that we use to prove Theorem 1. In the following, we investigate properties of this model, which allow us to simplify and strengthen Model (8).

2.1.2 Simplifying the alternative formulation

Observe that Model (8) is a mixed-integer non-linear programming model because it contains the product expressions $(1-x_u)\beta_{eu}$ and $(1-x_v)\beta_{ev}$ in Constraint (8b). As we show next, this non-linearity can be resolved by restricting the problem to a subspace that contains an optimal solution.

Theorem 3. *Let $\mathcal{G}=(\mathcal{V},\mathcal{E})$ be an undirected graph, let $\mathbf{c}\in\mathbb{R}^{\mathcal{V}}$, and let k be a positive integer such that the k-vertex cut problem is feasible. Then there exists an optimal solution $(\bar{\boldsymbol{x}},\bar{\boldsymbol{\lambda}},\bar{\boldsymbol{\beta}})$ to (8) such that $\bar{\lambda}_S\in\{0,1\}$ for all $S\in\mathscr{S}$ and $\bar{\beta}_{ew}=\bar{x}_w$ for all $e\in\mathcal{E}$, $w\in e$. In particular, if $\mathscr{K}$ is the set of connected components of the graph arising from $\mathcal{G}$ by removing the nodes in $\{v\in\mathcal{V}:\bar{x}_v=1\}$, then*

$$\bar{\lambda}_S = \begin{cases} 1, & \text{if } S\in\mathscr{K}, \\ 0, & \text{otherwise.} \end{cases}$$

Proof. Proof In Appendix A, we provide an explicit construction of such a solution. □ □

Corollary 4. *Let $\mathcal{G}=(\mathcal{V},\mathcal{E})$ be an undirected graph and let k be a positive integer. Let $\mathscr{C}$ be the family of all cliques in $\mathcal{G}$. Then, the constraints*

$$\sum_{S\in\mathscr{S}} \lambda_S = k, \qquad \sum_{S\in\mathscr{S}(v)} \lambda_S + x_v = 1, \quad v\in\mathcal{V}, \qquad \sum_{S\in\mathscr{S}(C)} \lambda_S \leq 1, \quad C\in\mathscr{C},$$

can be added to Model (8) without changing the optimal objective value.

Proof. Proof We defer the proof to Appendix A. □ □

Based on this theorem and corollary, we can simplify the model by dropping the non-linear terms from (8b), replacing the $\boldsymbol{\beta}$-variables by $\boldsymbol{x}$-variables, and adding additional constraints.

This results into the model

$$\min_{\substack{\boldsymbol{x}\in\{0,1\}^{\mathcal{V}}\\ \boldsymbol{\lambda}\geq\mathbf{0}}} \sum_{v\in\mathcal{V}} c_v x_v \tag{9a}$$

$$\sum_{S\in\mathscr{S}} (|S|-1)\lambda_S \leq n - \sum_{v\in\mathcal{V}} x_v - k, \tag{9b}$$

$$\sum_{S\in\mathscr{S}^{\subseteq}(e)} \lambda_S + x_u + x_v \geq 1, \qquad e=\{u,v\}\in\mathcal{E}, \tag{9c}$$

$$\sum_{S\in\mathscr{S}} \lambda_S = k, \tag{9d}$$

$$\sum_{S\in\mathscr{S}(v)} \lambda_S + x_v = 1, \qquad v\in\mathcal{V}, \tag{9e}$$

$$\sum_{S\in\mathscr{S}(C)} \lambda_S \leq 1, \qquad C\in\mathscr{C}. \tag{9f}$$

Comparing (9) and ILP_{E2}, observe that both models share the same variables (although the $\boldsymbol{\lambda}$-variables in Model (9) are continuous), objective, and constraints (9d), (9e), and (9f). To prove Theorem 1, we therefore need to show that the remaining constraints are redundant.

2.1.3 Proof of Theorem 1

To show redundancy of (9b), we sum (9e) for all $v\in\mathcal{V}$ to derive

$$|\mathcal{V}| - \sum_{v\in\mathcal{V}} x_v = \sum_{v\in\mathcal{V}} \sum_{S\in\mathscr{S}(v)} \lambda_S = \sum_{S\in\mathscr{S}} \sum_{v\in S} \lambda_S = \sum_{S\in\mathscr{S}} |S|\lambda_S.$$

We thus find

$$\sum_{S\in\mathscr{S}} (|S|-1)\lambda_S = \sum_{S\in\mathscr{S}} |S|\lambda_S - \sum_{S\in\mathscr{S}} \lambda_S = |\mathcal{V}| - \sum_{v\in\mathcal{V}} x_v - \sum_{S\in\mathscr{S}} \lambda_S \overset{(9d)}{\leq} |\mathcal{V}| - \sum_{v\in\mathcal{V}} x_v - k,$$

i.e., (9b) is redundant.

The redundancy of (9c) for edge $e=\{u,v\}$ can be shown by substituting x_u and x_v via (9e):

$$1 \leq \sum_{S\in\mathscr{S}^{\subseteq}(e)} \lambda_S + x_u + x_v = \sum_{S\in\mathscr{S}^{\subseteq}(e)} \lambda_S + 2 - \sum_{S\in\mathscr{S}(u)} \lambda_S - \sum_{S\in\mathscr{S}(v)} \lambda_S$$

$$\iff \quad 1 + \sum_{S\in\mathscr{S}^{\subseteq}(e)} \lambda_S \geq \sum_{S\in\mathscr{S}(u)} \lambda_S + \sum_{S\in\mathscr{S}(v)} \lambda_S = \sum_{S\in\mathscr{S}(e)} \lambda_S + \sum_{S\in\mathscr{S}^{\subseteq}(e)} \lambda_S$$

$$\iff \quad 1 \geq \sum_{S\in\mathscr{S}(e)} \lambda_S.$$

Since every edge is a clique, Constraint (9c) is implied by (9f). This concludes the proof. □

2.2 Strength of the LP relaxations

In the previous section, we have derived ILP_{E2} for the k-VCP, thus complementing the existing models ILP_{N} and ILP_{E1} from the literature. At first glance, however, it is unclear which model is best suited for solving the k-VCP. One common means to compare the strength of an ILP model is to assess the objective value of its LP relaxation: the closer the LP objective value is to the objective value of an optimal mixed-integer solution, the arguably more powerful the

bounding mechanism of branch-and-bound (or branch-and-price) is. We therefore compare the objective values of the LP relaxations of the different methods in the following.

Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ be an undirected graph that is endowed with vertex costs $\boldsymbol{c} \in \mathbb{R}^{\mathcal{V}}$, and let k be a positive integer. We then denote by $\zeta_{\mathrm{N}}(\mathcal{G}, \boldsymbol{c}, k)$, $\zeta_{\mathrm{E1}}(\mathcal{G}, \boldsymbol{c}, k)$, and $\zeta_{\mathrm{E2}}(\mathcal{G}, \boldsymbol{c}, k)$ the optimal objective value of the LP relaxations of $\mathrm{ILP}_{\mathrm{N}}$, $\mathrm{ILP}_{\mathrm{E1}}$, and $\mathrm{ILP}_{\mathrm{E2}}$, respectively. These LP models are called: LP_{N}, $\mathrm{LP}_{\mathrm{E1}}$, and $\mathrm{LP}_{\mathrm{E2}}$. Since $\mathrm{ILP}_{\mathrm{E1}}$ is in maximization form, $\zeta_{\mathrm{E1}}(\mathcal{G}, \boldsymbol{c}, k)$ is defined as the total vertex cost minus the optimal value of $\mathrm{LP}_{\mathrm{E1}}$, so that it is directly comparable with the optimal LP relaxation values of the other two models, namely LP_{N} and $\mathrm{LP}_{\mathrm{E2}}$, which are in minimization form.

Theorem 5. *Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ be an undirected graph, let $\boldsymbol{c} \in \mathbb{R}^{\mathcal{V}}$, and let k be a positive integer such that the k-vertex cut problem is feasible. Then,*

$$\zeta_{\mathrm{N}}(\mathcal{G}, \boldsymbol{c}, k) \leq \zeta_{\mathrm{E2}}(\mathcal{G}, \boldsymbol{c}, k) \qquad \text{and} \qquad \zeta_{\mathrm{E1}}(\mathcal{G}, \boldsymbol{c}, k) \leq \zeta_{\mathrm{E2}}(\mathcal{G}, \boldsymbol{c}, k).$$

That is to say, $\mathrm{LP}_{\mathrm{E2}}$ is not weaker than both, LP_{N} and $\mathrm{LP}_{\mathrm{E1}}$. In the remainder of this section, we provide a proof of this theorem. It is worth noticing that, throughout this section, we focus on feasible instances of the k-VCP, as the comparison of LP relaxation values is meaningful only in this case. The infeasible case is discussed separately in Section 3.1.1.

Lemma 6. *Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ be an undirected graph and let $\mathcal{F} = (\mathcal{V}(\mathcal{F}), \mathcal{E}(\mathcal{F})) \in \mathscr{F}_{\mathcal{G}}$. Let $(\boldsymbol{x}, \boldsymbol{\lambda})$ be a feasible solution to LP_{E2}. Then,*

$$\sum_{e \in \mathcal{E}(\mathcal{F})} \sum_{S \in \mathscr{S}^{\subseteq}(e)} \lambda_S \leq |\mathcal{V}| - \sum_{v \in \mathcal{V}} x_v - k.$$

Proof. Proof We observe

$$\begin{aligned}
\sum_{e \in \mathcal{E}(\mathcal{F})} \sum_{S \in \mathscr{S}^{\subseteq}(e)} \lambda_S &= \sum_{S \in \mathscr{S}} |\mathcal{E}(\mathcal{F})[S]|\, \lambda_S \leq \sum_{S \in \mathscr{S}} (|S| - 1)\, \lambda_S = \sum_{S \in \mathscr{S}} |S| \lambda_S - \sum_{S \in \mathscr{S}} \lambda_S \\
&= \sum_{S \in \mathscr{S}} \sum_{v \in S} \lambda_S - \sum_{S \in \mathscr{S}} \lambda_S = \sum_{v \in \mathcal{V}} \sum_{S \in \mathscr{S}(v)} \lambda_S - \sum_{S \in \mathscr{S}} \lambda_S \overset{(4c)}{=} \sum_{v \in \mathcal{V}} (1 - x_v) - \sum_{S \in \mathscr{S}} \lambda_S \\
&= |\mathcal{V}| - \sum_{v \in \mathcal{V}} x_v - \sum_{S \in \mathscr{S}} \lambda_S \overset{(4b)}{=} |\mathcal{V}| - \sum_{v \in \mathcal{V}} x_v - k,
\end{aligned}$$

where the first inequality holds, because a forest contains at most $|S| - 1$ edges from a set S and $\boldsymbol{\lambda}$ is non-negative. □ □

Proposition 7. *Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ be an undirected graph, let $\boldsymbol{c} \in \mathbb{R}^{\mathcal{V}}$, and let k be a positive integer such that the k-vertex cut problem is feasible. Then, $\zeta_{\mathrm{N}}(\mathcal{G}, \boldsymbol{c}, k) \leq \zeta_{\mathrm{E2}}(\mathcal{G}, \boldsymbol{c}, k)$.*

Proof. Proof Let $(\boldsymbol{x}, \boldsymbol{\lambda})$ be feasible for $\mathrm{LP}_{\mathrm{E2}}$. It suffices to prove that $\boldsymbol{x}$ satisfies (3b). To this end, let $\mathcal{F} = (\mathcal{V}(\mathcal{F}), \mathcal{E}(\mathcal{F})) \in \mathscr{F}_{\mathcal{G}}$. Recall that, for all $e \in \mathcal{E}$, Inequality (9c) is redundant for $\mathrm{LP}_{\mathrm{E2}}$. Hence, these inequalities are valid and summing them for all $\{u, v\} \in \mathcal{E}(\mathcal{F})$, we obtain

$$\begin{aligned}
|\mathcal{E}(\mathcal{F})| &\leq \sum_{\{u,v\} \in \mathcal{E}(\mathcal{F})} \left(\sum_{S \in \mathscr{S}^{\subseteq}(\{u,v\})} \lambda_S + x_u + x_v \right) \\
&= \sum_{v \in \mathcal{V}} \deg_{\mathcal{F}}(v) x_v + \sum_{e \in \mathcal{E}(\mathcal{F})} \sum_{S \in \mathscr{S}^{\subseteq}(e)} \lambda_S \overset{\text{Lemma 6}}{\leq} |\mathcal{V}| - k + \sum_{v \in \mathcal{V}} (\deg_{\mathcal{F}}(v) - 1) x_v.
\end{aligned}$$

Hence, $\boldsymbol{x}$ is feasible for LP_{N}, which proves the assertion. □ □

Proposition 8. *Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ be an undirected graph, let $\boldsymbol{c} \in \mathbb{R}^{\mathcal{V}}$, and let k be a positive integer such that the k-vertex cut problem is feasible. Then, $\zeta_{\mathrm{E1}}(\mathcal{G}, \boldsymbol{c}, k) \leq \zeta_{\mathrm{E2}}(\mathcal{G}, \boldsymbol{c}, k)$. Moreover, if $\mathscr{C}' = \mathscr{C}$ for defining (2d) in ILP$_{E1}$, then $\zeta_{\mathrm{E1}}(\mathcal{G}, \boldsymbol{c}, k) = \zeta_{\mathrm{E2}}(\mathcal{G}, \boldsymbol{c}, k)$.*

Proof. Proof To prove the assertion, we eliminate the $\boldsymbol{x}$-variables from $\mathrm{LP}_{\mathrm{E2}}$. By substituting x_v via (4c), we obtain

$$\min_{\boldsymbol{\lambda} \geq \boldsymbol{0}} \left\{ \sum_{v \in \mathcal{V}} c_v \left(1 - \sum_{S \in \mathscr{S}(v)} \lambda_S\right) : \sum_{S \in \mathscr{S}} \lambda_S = k, \quad \sum_{S \in \mathscr{S}(C)} \lambda_S \leq 1,\ C \in \mathscr{C}, \quad \sum_{S \in \mathscr{S}(v)} \lambda_S \leq 1,\ v \in \mathcal{V} \right\}.$$

Since (4d) is not weaker than (2d), $\boldsymbol{\lambda}$ is feasible for $\mathrm{LP}_{\mathrm{E1}}$.

Moreover, note that the objective function simplifies to

$$\sum_{v \in \mathcal{V}} \left(c_v - c_v \sum_{S \in \mathscr{S}(v)} \lambda_s\right) = \sum_{v \in \mathcal{V}} c_v - \sum_{S \in \mathscr{S}} \sum_{v \in S} c_v \lambda_s$$

and is thus equivalent to the objective of $\mathrm{ILP}_{\mathrm{E1}}$. Consequently, $\zeta_{\mathrm{E1}}(\mathcal{G}, \boldsymbol{c}, k) \leq \zeta_{\mathrm{E2}}(\mathcal{G}, \boldsymbol{c}, k)$.

If $\mathscr{C}' = \mathscr{C}$, let $\boldsymbol{\lambda}$ be a feasible solution to $\mathrm{LP}_{\mathrm{E1}}$. Then, $(\boldsymbol{x}, \boldsymbol{\lambda})$, where $x_v = 1 - \sum_{S \in \mathscr{S}(v)} \lambda_S$, $v \in \mathcal{V}$, is easily seen to be feasible to $\mathrm{LP}_{\mathrm{E2}}$. Since arguments analogous to the above show that both solutions have the same objective value, we conclude $\zeta_{\mathrm{E2}}(\mathcal{G}, \boldsymbol{c}, k) \leq \zeta_{\mathrm{E1}}(\mathcal{G}, \boldsymbol{c}, k)$, and thus $\zeta_{\mathrm{E2}}(\mathcal{G}, \boldsymbol{c}, k) = \zeta_{\mathrm{E1}}(\mathcal{G}, \boldsymbol{c}, k)$. □ □

Combining these results thus yields Theorem 5.

3 A branch-and-price algorithm to solve the new extended formulation

In this section, we describe the branch-and-price framework that we use to solve $\mathrm{ILP}_{\mathrm{E2}}$. Since $\mathrm{ILP}_{\mathrm{E2}}$ has exponentially many variables and possibly exponentially many constraints, we do not initialize it with all $\boldsymbol{\lambda}$-variables and constraints of type (4d). Instead, we describe which subsets of $\boldsymbol{\lambda}$-variables and constraints are used to initialize Model $\mathrm{ILP}_{\mathrm{E2}}$ to obtain the so-called restricted master problem (RMP) in Section 3.1.1. We continue the discussion by analyzing the pricing problem at the root node (Section 3.1.2), which is followed by describing our branching scheme and how it is respected by the pricing problem (Section 3.1.3). Afterwards, we explain the separation problem for (4d) (Section 3.1.4). We conclude the section by providing symmetry handling techniques (Section 3.2) and further algorithmic enhancements (Section 3.3) for accelerating the BP algorithm.

3.1 The branch-and-price framework

In the following, we describe the building blocks of our branch-and-price framework to solve $\mathrm{ILP}_{\mathrm{E2}}$. Before doing so, we observe that all equality constraints of $\mathrm{ILP}_{\mathrm{E2}}$ can be relaxed into inequalities and still yield a correct model of the k-VCP. Although relaxing these constraints can, in principle, weaken the LP relaxation, we observed that the model with relaxed constraints performs better in practice, and we discuss possible explanations for this behavior later in Section 4.2.

Proposition 9. *Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ be an undirected graph with vertex costs $\boldsymbol{c} \in \mathbb{R}^{\mathcal{V}}$ and let k be a positive integer. Moreover, let $\mathscr{C}'$ be an edge-covering family of cliques of $\mathcal{G}$. Then,*

$$(ILP^{\star}_{E2}) \qquad \min_{\substack{\boldsymbol{x}\in\{0,1\}^{\mathcal{V}} \\ \boldsymbol{\lambda}\geq\boldsymbol{0}}} \sum_{v\in\mathcal{V}} c_v\, x_v \tag{10a}$$

$$\sum_{S\in\mathscr{S}} \lambda_S \geq k, \tag{10b}$$

$$\sum_{S\in\mathscr{S}(v)} \lambda_S + x_v \geq 1, \qquad v \in \mathcal{V}, \tag{10c}$$

$$\sum_{S\in\mathscr{S}(C)} \lambda_S \leq 1, \qquad C \in \mathscr{C}'. \tag{10d}$$

is a correct model of the k-VCP.

The branch-and-price framework that we discuss in the following is based on Model $\text{ILP}^{\star}_{\text{E2}}$.

Proof. Proof Based on Theorem 1, we can relax the integrality requirement on the $\boldsymbol{\lambda}$-variables. To conclude the proof, we first show that it is possible to relax the equality in Constraint (4b) and to define (4d) only in terms of an edge-covering family of cliques rather than all cliques. Afterwards, we also argue that (4c) can be relaxed.

Based on Theorem 3 and Corollary 4, replacing $\boldsymbol{\beta}$-variables in Model (8) by $\boldsymbol{x}$-variables and augmenting the resulting model by $\sum_{S\in\mathscr{S}} \lambda_S \geq k$ and $\sum_{S\in\mathscr{S}(v)} \lambda_S + x_v = 1$ for all $v \in \mathcal{V}$ is a correct model of the k-VCP. Following the same arguments as in Section 2.1.3, Constraint (8b) is redundant in the augmented model. Moreover, for every $\{u, v\} \in \mathcal{E}$, we substitute x_u and x_v in (8c) via (9e) to obtain

$$1 \leq \sum_{S\in\mathscr{S}^{\subseteq}(\{u,v\})} \lambda_S + \left(1 - \sum_{S\in\mathscr{S}(u)} \lambda_S\right) + \left(1 - \sum_{S\in\mathscr{S}(v)} \lambda_S\right)$$

$$\iff \quad 1 + \sum_{S\in\mathscr{S}^{\subseteq}(\{u,v\})} \lambda_S \geq \sum_{S\in\mathscr{S}(u)} \lambda_S + \sum_{S\in\mathscr{S}(v)} \lambda_S = \sum_{S\in\mathscr{S}(\{u,v\})} \lambda_S + \sum_{S\in\mathscr{S}^{\subseteq}(\{u,v\})} \lambda_S.$$

That is, the *edge constraint* $\sum_{S\in\mathscr{S}(\{u,v\})} \lambda_S \leq 1$ is an equivalent representation of Constraint (8c) in the augmented model. Since the clique constraints (9f) are valid for the augmented model by Corollary 4 and such a constraint for a clique C dominates the edge constraints for edges $\{u, v\} \subseteq C$, it is sufficient to define (4d) in terms of an edge-covering family of cliques.

To conclude the proof, we show that we can relax equality in (4c). Let $(\boldsymbol{x}, \boldsymbol{\lambda})$ be an optimal solution of (10). The assertion follows if we can establish that $(\boldsymbol{x}, \boldsymbol{\lambda})$ satisfies $\sum_{S\in\mathscr{S}(v)} \lambda_S + x_v = 1$. This is indeed the case, since every vertex $v \in \mathcal{V}$ is contained in a clique $C \in \mathscr{C}'$. Hence, (10d) and non-negativity of the $\boldsymbol{\lambda}$-variables imply $\sum_{S\in\mathscr{S}(v)} \lambda_S \leq 1$. If there was a $v \in \mathcal{V}$ such that $\sum_{S\in\mathscr{S}(v)} \lambda_S + x_v > 1$, we could improve the objective value by decreasing x_v because $c_v > 0$. This, however, contradicts optimality of $(\boldsymbol{x}, \boldsymbol{\lambda})$. □ □

3.1.1 Initial sets of variables and constraints

$\text{ILP}^{\star}_{\text{E2}}$ contains two families of variables, the $\boldsymbol{x}$-variables and the $\boldsymbol{\lambda}$-variables. All the $|\mathcal{V}|$ variables of the first type are added to the RMP. For the second type of variables, we follow Cornaz et

al. (2019) and only add $\lambda_{\{v\}}$, $v \in \mathcal{V}$, to the RMP. Using the same arguments as Cornaz et al. (2019), this choice guarantees that the RMP is feasible whenever the k-VCP admits a feasible solution. To make this article self-contained, we provide the proof of this statement.

Lemma 10. *Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ be an undirected graph and let k be a positive integer. Then, $\mathcal{G}$ admits a k-vertex cut if and only if $\mathcal{G}$ admits a stable set of size k. In particular, if $S \subseteq \mathcal{V}$ is a stable set of size k, then $\mathcal{V} \setminus S$ is a k-vertex cut.*

Proof. Proof On the one hand, if $\mathcal{G}$ admits a k-vertex cut, let $D \subseteq \mathcal{V}$ be a set of vertices whose removal disconnects $\mathcal{G}$ into at least $\ell \geq k$ connected components $K_1, \ldots, K_\ell$. Then, for all distinct $i, j \in \{1, \ldots, \ell\}$, no pair of vertices $v_i \in K_i$ and $v_j \in K_j$ is adjacent. Thus, by selecting a vertex from each of the connected components, we create a stable set of size $\ell \geq k$.

On the other hand, if $\mathcal{G}$ admits a stable set S of size k, then $\mathcal{V} \setminus S$ is a k-vertex cut of $\mathcal{G}$. □ □

Based on Theorem 1 and Lemma 10 we thus obtain the following proposition.

Proposition 11. *Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ be an undirected graph with vertex costs $\boldsymbol{c} \in \mathbb{R}^{\mathcal{V}}$ and let k be a positive integer. If $\mathcal{G}$ admits a k-vertex cut, then the RMP for $ILP^{\star}_{E2}$ for the choice $\mathscr{S} = \{\{v\} : v \in \mathcal{V}\}$ is feasible.*

That is, whenever $\mathcal{G}$ admits a k-vertex cut, the restricted master problem is always feasible. Moreover, if the restricted master problem is infeasible, we can conclude that $\mathcal{G}$ does not admit a k-vertex cut.

The RMP contains the Constraint (10b) and the $|\mathcal{V}|$ Constraints (10c). To construct the edge-covering family of cliques $\mathscr{C}'$ used in Constraints (10d), we propose the following constructive algorithm that is an iterative procedure. At each step, it builds one inclusion-wise maximal clique and maintains a set of uncovered edges, which is initially equal to the original edge set. To build a clique C, the algorithm selects an uncovered edge $\{u, v\}$ and initializes C as the set containing its two endpoints. It then repeatedly attempts to enlarge C by adding a vertex $w \in \mathcal{V} \setminus C$ such that w is adjacent to every vertex already in C. When no further vertex can be added, the clique C is inclusion-wise maximal. The clique is then added to the family $\mathscr{C}'$, and all edges whose endpoints both belong to C are removed from the set of uncovered edges. The algorithm terminates when no uncovered edges remain. Both edges and vertices are processed according to the natural ordering induced by the input instance. By construction, this constructive algorithm produces at most $|\mathcal{E}|$ cliques.

3.1.2 The pricing problem

We solve an LP relaxation of $\text{ILP}^{\star}_{\text{E2}}$ via the RMP, by relaxing the integrality constraints on the $\boldsymbol{x}$-variables to non-negativity constraints, and by considering only a subset $\overline{\mathscr{S}} \subseteq \mathscr{S}$ for $\boldsymbol{\lambda}$-variables.

Note that we do not impose the upper bound $x_v \leq 1$, $v \in \mathcal{V}$, in the RMP, because in every *optimal* solution $x_v \leq 1$ will hold, which is a consequence of the last paragraph of the proof of Proposition 9. Dropping the upper bounds on the $\boldsymbol{x}$-variables does not affect the pricing problem.

To find an optimal solution of the LP relaxation of $\text{ILP}^{\star}_{\text{E2}}$, it might not be enough to solve the RMP for the selected set $\overline{\mathscr{S}}$. Given a solution $(\boldsymbol{x}, \boldsymbol{\lambda})$ of the RMP, we therefore need to decide whether there exists a variable λ_S for some $S \in \mathscr{S} \setminus \overline{\mathscr{S}}$ with negative reduced cost. This problem is called the pricing problem and coincides with the separation problem of a dual solution associated with $(\boldsymbol{x}, \boldsymbol{\lambda})$. Let $\sigma \geq 0$ be the dual variable associated with (10b), let $\mu_v \geq 0$, $v \in \mathcal{V}$, be the dual variables associated with (10c), and let $\pi_C \geq 0$, $C \in \mathscr{C}'$, be the dual variables associated with (10d). The dual constraints associated to the $\boldsymbol{\lambda}$-variables are

$$\sigma + \sum_{v \in S} \mu_v - \sum_{\substack{C \in \mathscr{C}': \\ S \cap C \neq \emptyset}} \pi_C \leq 0, \qquad S \in \overline{\mathscr{S}}.$$

Given a dual solution $(\sigma, \boldsymbol{\mu}, \boldsymbol{\pi})$, the pricing problem for $\boldsymbol{\lambda}$-variables is to decide if there exists $S \in \mathscr{S} \setminus \overline{\mathscr{S}}$ such that

$$\sigma + \sum_{v \in S} \mu_v - \sum_{\substack{C \in \mathscr{C}': \\ S \cap C \neq \emptyset}} \pi_C > 0 \qquad \Longleftrightarrow \qquad \sum_{v \in S} \mu_v - \sum_{\substack{C \in \mathscr{C}': \\ S \cap C \neq \emptyset}} \pi_C > -\sigma.$$

Deciding whether such S exists is therefore equivalent to solving the optimization problem

$$\max_{S \subseteq \mathcal{V}} \left\{ \sum_{v \in S} \mu_v - \sum_{\substack{C \in \mathscr{C}': \\ S \cap C \neq \emptyset}} \pi_C : |S| \geq 1 \right\}. \tag{11}$$

Indeed, there exists a violated dual constraint if and only if the maximum takes a value that is larger than $-\sigma$.

In the following, we discuss how to solve the pricing problem. To better understand its structure, it is useful to express it as an ILP. To model the selected set S, let φ_v, $v \in \mathcal{V}$, be a binary variable that takes value 1 if and only if v belongs to S. Moreover, for every clique $C \in \mathscr{C}'$, let ψ_C be a binary variable that takes value 1 whenever the generated subset S intersects clique C (though the converse does not necessarily hold). An ILP model of the pricing problem, for a given dual solution $(\sigma, \boldsymbol{\mu}, \boldsymbol{\pi})$, is then

$$\max_{\substack{\boldsymbol{\varphi} \in \{0,1\}^{\mathcal{V}} \\ \boldsymbol{\psi} \in \{0,1\}^{\mathscr{C}'}}} \left\{ \sum_{v \in \mathcal{V}} \mu_v \varphi_v - \sum_{C \in \mathscr{C}'} \pi_C \psi_C : \quad \sum_{v \in \mathcal{V}} \varphi_v \geq 1, \quad \varphi_v \leq \psi_C, \; C \in \mathscr{C}', \; v \in C \right\}. \tag{12}$$

The objective function in (12) maximizes the violation of the dual constraint. The first constraint (*cardinality constraint*) prevents selecting the empty set, while the second family of constraints (*precedence constraints*) ensures that ψ_C takes value 1 whenever the chosen set S contains a vertex belonging to the clique C.

The pricing problem (11) can be solved in polynomial time: as shown by Cornaz et al. (2019), if the cardinality constraint is removed from (12), the resulting ILP has a totally unimodular constraint matrix. Indeed, the precedence constraints define a bipartite incidence structure between vertices and cliques, while all remaining conditions are simple (integer) bounds. Consequently, the LP relaxation of the reduced model is integral and can be solved efficiently. However, removing the cardinality constraint may yield the trivial solution $S = \emptyset$. To enforce $S \neq \emptyset$, one can solve the LP relaxation of the reduced model once for each vertex $v \in \mathcal{V}$, adding the fixing constraint $\varphi_v = 1$. Total unimodularity is preserved, so each resulting LP again admits an integral optimal solution, which corresponds to a valid non-empty set S_v. Among all such sets,

one achieving the largest objective value is selected as the solution of the pricing problem. Since a polynomial number of LPs is solved and each LP can be solved in polynomial time, the pricing routine is polynomial as well. By the classical equivalence between optimization and separation (Grötschel et al. 1981), this also implies that the LP relaxation of $\text{ILP}^{\star}_{\text{E2}}$ can be solved in polynomial time.

Complementing the LP techniques for solving the pricing problem, we discuss a combinatorial algorithm that is faster in practice. A variant of this algorithm was already used by Cornaz et al. (2019) for ILP_{E1}. To make this paper self-contained and since their explanation of this approach omits many details, we provide a detailed explanation. Moreover, we provide some important algorithmic improvements that fully preserve the use of the combinatorial algorithm for the pricing phase. We exploit a result by Barahona & Jensen (1998) who describe how to optimize a linear objective subject to precedence constraints (like the ones in (12)). Their idea is to solve the maximization problem by computing a minimum cut in an auxiliary network $\mathcal{N} = (\mathcal{V}(\mathcal{N}), \mathcal{A}(\mathcal{N}))$. Note that this approach neglects the cardinality constraint and we will discuss how to enforce it at the end of the section.

The vertex set of the network is given by $\mathcal{V}(\mathcal{N}) = \{s, t\} \cup \mathcal{V} \cup \mathscr{C}'$, where s and t are the source and sink of the network. The arc set $\mathcal{A}(\mathcal{N})$ consists of (i) the arcs (s, v) with capacity μ_v for all $v \in \mathcal{V}$, (ii) the arcs (C, t) with capacity π_C for all $C \in \mathscr{C}'$, and (iii) the arcs (v, C) with infinite capacity for all $C \in \mathscr{C}'$ and $v \in C$. An example of this construction is shown in Figure 3. An s–t cut in $\mathcal{N}$ is a partition (S', T') of $\mathcal{V}(\mathcal{N})$ such that $s \in S'$ and $t \in T'$. The arcs crossing a cut (S', T') are the arcs $\Upsilon(S', T') = \{(u, v) \in \mathcal{A}(\mathcal{N}) : u \in S',\ v \in T'\}$; the capacity of the cut is the total capacity of its crossing arcs. Barahona & Jensen (1998) noted that there is a one-to-one correspondence between s–t cuts with finite capacity and feasible solutions of (12) without the cardinality constraint. Indeed, given a cut (S', T') of finite capacity, the corresponding set $S = S(S', T')$ is $\{v \in \mathcal{V} : (s, v) \notin \Upsilon(S', T')\}$. The capacity of this cut is

$$\sum_{v \in \mathcal{V} \setminus S} \mu_v + \sum_{\substack{C \in \mathscr{C}' : \\ C \cap S \neq \emptyset}} \pi_C,$$

Thus, since

$$\sum_{v \in S} \mu_v - \sum_{\substack{C \in \mathscr{C}' : \\ C \cap S \neq \emptyset}} \pi_C = \sum_{v \in \mathcal{V}} \mu_v - \sum_{v \in \mathcal{V} \setminus S} \mu_v - \sum_{\substack{C \in \mathscr{C}' : \\ C \cap S \neq \emptyset}} \pi_C = \sum_{v \in \mathcal{V}} \mu_v - \left(\sum_{v \in \mathcal{V} \setminus S} \mu_v + \sum_{\substack{C \in \mathscr{C}' : \\ C \cap S \neq \emptyset}} \pi_C \right),$$

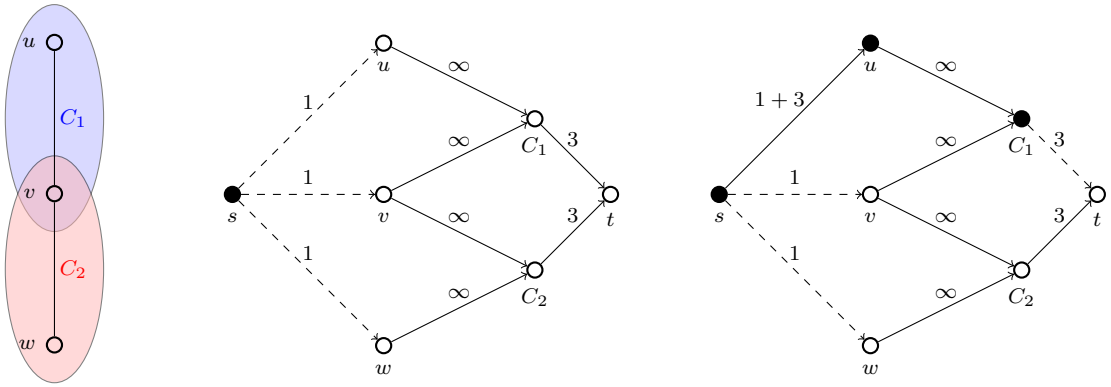
an optimal solution of (12) without the cardinality constraint can be found by computing a minimum cut in the auxiliary network $\mathcal{N}$.

To solve the pricing problem, we propose a two stage procedure. In the first stage, we find a minimum cut (S', T') in the network defined above and extract the set $S = S(S', T')$. If $S \neq \emptyset$

The second stage is entered only when $S = \emptyset$ and therefore

$$-\sigma < \sum_{v \in S} \mu_v - \sum_{\substack{C \in \mathscr{C}' : \\ C \cap S \neq \emptyset}} \pi_C = 0.$$

We then need to decide if there exists an alternative minimum cut (S'', T'') with $S(S'', T'') \neq \emptyset$ whose objective value in (12) is greater than $-\sigma$. A situation in which this happens is illustrated in Figure 3a. Here, a cut corresponding to $S = \emptyset$ is produced although there exist a variable with a negative reduced cost associated to the non-empty set $S = \{u\}$. To determine such type



(a) $S = \emptyset$. (b) $S = \{u\}$.

Figure 3: Illustration of the auxiliary network for solving the pricing problem on a graph with three vertices u, v, w and two maximal cliques, $C_1 = \{u, v\}$ (blue) and $C_2 = \{v, w\}$ (red). Arc capacities are given next to the arcs.

of variables, it turns out that it is sufficient to solve $|\mathcal{V}|$ auxiliary minimum cut problems as we explain next.

Assume there exists a set $S \subseteq \mathcal{V}$ such that λ_S has negative reduced cost. Let us assume that we had access to a set S of minimum reduced cost. We then could construct an alternative dual solution $(\bar{\sigma}, \bar{\boldsymbol{\mu}}, \bar{\boldsymbol{\pi}})$ such that the optimal value in (12) remains invariant by selecting $\bar{v} \in S$ and defining

$$\bar{\sigma} = 0, \qquad \bar{\mu}_v = \begin{cases} \mu_{\bar{v}} + \sigma, & \text{if } v = \bar{v}, \\ \mu_v, & \text{otherwise,} \end{cases} \qquad \bar{\boldsymbol{\pi}} = \boldsymbol{\pi}.$$

Indeed, for every non-empty subset $S' \subseteq \mathcal{V}$ that contains $\bar{v}$, the objective value in (12) remains unchanged and, if $\bar{v} \notin S'$, the objective value drops by σ. Therefore, the set S that we started with is still an optimizer of (12). Since the second stage is only entered when $\sigma > 0$, this means that solving the minimum cut problem on the auxiliary network $\mathcal{N}$ for the dual solution $(\bar{\sigma}, \bar{\boldsymbol{\mu}}, \bar{\boldsymbol{\pi}})$ will result in finding a set S that contains $\bar{v}$.

In practice, of course, we do not know a set S of minimum reduced cost. But nevertheless we can adopt the ideas above. For each $v \in \mathcal{V}$, we solve a minimum cut problem in the network $\mathcal{N}$, where we increase the capacity of arc (s, v) by σ. If one of these minimum cuts yields a non-empty set S such that its associated variable λ_S has negative reduced cost, we add it as an improving column to the RMP. Otherwise, no variable with negative reduced cost exists.

Figure 3 illustrates the auxiliary network on an example graph with three vertices u, v, w, whose edge-covering clique family consists of two cliques, $C_1 = \{u, v\}$ (highlighted in blue) and $C_2 = \{v, w\}$ (highlighted in red). In the auxiliary network, black-filled nodes denote the source side S' of the min-cut, white nodes denote the sink side T', and dashed arcs are the arcs of $\Upsilon(S', T')$ crossing the cut. In this example, the dual solution is $\mu_u = \mu_v = \mu_w = 1$, $\pi_{C_1} = \pi_{C_2} = 3$ (defining the arc capacities shown next to the arcs), and $\sigma = 3$. In Figure 3a, the auxiliary network yields a min-cut associated to the arcs (s, u), (s, v), (s, w), with capacity 3, producing the solution $S = \emptyset$. Augmenting the capacity of arc (s, u) by $\sigma = 3$ (Figure 3b) changes the min-cut: it is now associated to the arcs (s, v), (s, w), and (C_1, t) with total capacity $5 <$

$1 + 4 = 6$, yielding the solution $S = \{u\}$.

3.1.3 The branching scheme

A central component of a branch-and-price algorithm is the branching scheme. For ILP_{E1}, Cornaz et al. (2019) suggest at two-step branching procedure. In $\text{ILP}^{\star}_{\text{E2}}$, however, the $\boldsymbol{\lambda}$-variables are continuous and it is not necessary to branch on them. Instead, we only branch on the $\boldsymbol{x}$-variables. We use the reliable pseudo cost branching rule (Achterberg 2007). When solving the RMP at a node of the branching tree that is different from the root node, the branching decisions on the $\boldsymbol{x}$-variables must be respected by the pricing problem.

We describe how the branching decisions are incorporated into the combinatorial pricing algorithm. If there exists a vertex $v \in \mathcal{V}$ such that $x_v = 1$ holds due to branching, then v is contained in the vertex cut and, therefore, cannot be included in the generated set S. This decision can be incorporated into the pricing problem by adding the arc (v, t) of infinite capacity to the network used for solving the pricing problem. Due to the infinite capacity, this arc will never be contained in a minimum cut, thus representing the branching decision $x_v = 1$.

To model the branching decision $x_v = 0$, we need to make sure that v is not contained in the neighborhood

$$N(S) = \{w \in \mathcal{V} \setminus S : \exists u \in S \text{ with } \{u, w\} \in \mathcal{E}\}$$

of the set S that is generated by the pricing problem. Indeed, since $x_v = 0$, vertex v needs to be contained in some cluster. But if $v \in N(S)$ this would mean that v is not contained in S and the cluster S' containing v is connected with S—a contradiction to a k-vertex cut. To incorporate the neighborhood condition into the pricing problem, we can add arcs (w, v) with infinite capacity for all $w \in \mathcal{V}$ with $\{v, w\} \in \mathcal{E}$. Thus, in a minimum cut, it is impossible to include a neighbor of v into S while excluding v from S.

For the sake of completeness, we now describe how the ILP-based pricing problem (12) can be modified according to the branching decisions. In case x_v is fixed to 1, vertex v belongs to the vertex cut and cannot be part of any generated cluster S; this can be modeled by fixing $\varphi_v = 0$. Conversely, if x_v is fixed to 0, a neighbor w of vertex v can belong to S only if v is in S as well; this can be modeled by including the following additional precedence constraints: $\varphi_w \leq \varphi_v$, for all $w \in N(v)$. In both cases, the total unimodularity of the constraint matrix is preserved.

3.1.4 The separation problem for the clique-partitioning constraints (4d)

In principle, it is possible to strengthen formulation $\text{ILP}^{\star}_{\text{E2}}$ by explicitly separating violated constraints of type (10d), rather than restricting ourselves to the inequalities associated with the initial edge-covering family of cliques $\mathscr{C}'$. Given a fractional solution $(\boldsymbol{x}, \boldsymbol{\lambda})$ of the LP relaxation, the corresponding separation problem asks whether there exists a clique $C \subseteq \mathcal{V}$ such that $\sum_{S \in \mathscr{S}(C)} \lambda_S > 1$, and, if so, to identify a clique with maximum violation. In order to fully assess the strength of formulation $\text{ILP}^{\star}_{\text{E2}}$, we also implemented a branch-and-price-and-cut algorithm in which the clique partitioning constraints (10d) are separated exactly at the root node. This setting allows us to exploit the full potential of these inequalities and to benchmark the best bound that can be obtained from $\text{ILP}^{\star}_{\text{E2}}$. The computational results reported in Appendix F indicate that, for the instances considered, a comparable dual bound can already

be achieved by adding only the constraints associated with the edge-covering family of cliques $\mathscr{C}'$ introduced in Section 3.1. In other words, the initial edge-covering family of cliques $\mathscr{C}'$ suffices to obtain a bound of very high quality, which is in most cases almost indistinguishable from that obtained with exact separation at the root. Since the corresponding separation problem is $\mathcal{NP}$-hard and adds a computational overhead, our final algorithm therefore does not include the exact separation of clique partitioning constraints and relies exclusively on the inequalities generated from $\mathscr{C}'$. Further details, including the branch-and-price-and-cut performance and the proof of $\mathcal{NP}$-hardness of the separation problem, are provided in Appendix F.

3.2 Symmetry handling techniques

To further enhance solving $\text{ILP}^\star_{\text{E2}}$, we observe that the k-vertex cut problem admits structurally equivalent solutions if the underlying graph is symmetric. Consider, for example, the minimum 2-vertex cut problem on a cycle of length $n \geq 4$, and assume its vertices are consecutively labeled by $1, \dots, n$. Then, a family of solutions is given by $\{1,3\}, \{2,4\}, \dots, \{n-2, n\}, \{1, n-1\}$, i.e., one removes two vertices which are at distance 2 from each other. Although these solutions are structurally identical, they look differently to a solver because they are encoded by different variables. In the MIP literature, it is well-known that, when not handled appropriately, the presence of such symmetries can considerably hinder solving integer programs by branch-and-bound (Pfetsch & Rehn 2019). For this reason, a wealth of techniques has been developed to handle symmetries within branch-and-bound in the last two decades. The main goal of these techniques is to prune parts of the search tree when it is known that another part contains symmetric information, which arguably accelerates branch-and-bound. The branch-and-price literature knows problem specific symmetry breaking and, of course, elimination of model symmetry by aggregation of identical subproblems. However, we may additionally have symmetry in the input instance. This can be exploited for pruning the branch-and-price tree using the generic symmetry handling machinery built into solvers. In the following, we do exactly this when solving $\text{ILP}^\star_{\text{E2}}$. To the best of our knowledge, our application of symmetry handling (on what can be called the original variables) is the first of its kind in a branch-and-price context.

To formally introduce our approach, we require some notation. A permutation γ of $\{1, \dots, n\}$ acts on an n-dimensional vector $\boldsymbol{y} \in \mathbb{R}^n$ by permuting its indices, i.e., $\gamma(\boldsymbol{y}) \coloneqq (y_{\gamma^{-1}(1)}, \dots, y_{\gamma^{-1}(n)})$. Such a permutation γ defines a *symmetry* of the integer program $\min_{\boldsymbol{y} \in \mathbb{Z}^n} \{\boldsymbol{c}^\top \boldsymbol{y} : \boldsymbol{A}\boldsymbol{y} \leq \boldsymbol{b}\}$, where $\boldsymbol{A}$ is a matrix and $\boldsymbol{b}$ and $\boldsymbol{c}$ are vectors of suitable dimensions, if for all $\boldsymbol{y} \in \mathbb{Z}^n$ we have that $\boldsymbol{A}\boldsymbol{y} \leq \boldsymbol{b}$ holds if and only if $\boldsymbol{A}\gamma(\boldsymbol{y}) \leq \boldsymbol{b}$ holds, and $\boldsymbol{c}^\top \gamma(\boldsymbol{y}) = \boldsymbol{c}^\top \boldsymbol{y}$. That is, γ preserves both feasibility and the objective value of a solution. In the following, we want to derive a class of symmetries for $\text{ILP}^\star_{\text{E2}}$. As mentioned above, a natural class of symmetries is given by symmetries of the underlying graph. To formally define graph symmetries, let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ be an undirected graph with vertex costs $\boldsymbol{c} \in \mathbb{R}^V$. An *automorphism* of $\mathcal{G}$ is a map $\pi \colon \mathcal{V} \to \mathcal{V}$ such that $c_v = c_{\pi(v)}$ for all $v \in \mathcal{V}$ and $\{\pi(u), \pi(v)\} \in \mathcal{E}$ for all $\{u, v\} \in \mathcal{E}$. That is, π can be considered a relabeling of the vertices of $\mathcal{G}$ that preserves vertex costs and adjacency.

Proposition 12. *Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ be an undirected graph with vertex costs $\boldsymbol{c} \in \mathbb{R}^V$, let γ be an automorphism of $\mathcal{G}$, and let k be a positive integer. Let $\boldsymbol{\lambda} = (\lambda_S)_{S \in \mathscr{S}}$. Then, γ defines a symmetry of $\text{ILP}^\star_{E2}$, where $\gamma(\boldsymbol{x}) = (x_{\gamma^{-1}(1)}, \dots, x_{\gamma^{-1}(n)})$ and $\gamma(\boldsymbol{\lambda}) = (\lambda_{\gamma^{-1}(S)})_{S \in \mathscr{S}}$,*

where $\gamma^{-1}(S) = \{\gamma^{-1}(v) : v \in S\}$.

For a proof, we refer the reader to Appendix A.

Knowing a class of symmetries of $\text{ILP}^{\star}_{\text{E2}}$, we can apply state-of-the-art methods for handling them. A common feature of most existing symmetry handling techniques is to define a lexicographic order on the variable space and to enforce that only those solutions can be computed that are lexicographically maximal in their class of symmetric solutions (van Doornmalen & Hojny 2025). In a branch-and-price context, however, the challenge arises that not all variables are known explicitly. We therefore suggest to define the lexicographic order only on the $\boldsymbol{x}$-variables and ignore the $\boldsymbol{\lambda}$-variables for symmetry considerations.

We have experimented with two approaches for handling symmetries. For both approaches, we assume that we are given access to a set Γ of automorphisms of $\mathcal{G}$. To explain the differences between these approaches, let us assume $\mathcal{V} = \{1, \dots, n\}$.

The first approach defines a global lexicographic order $\geq_{\text{lex}}$, where $\boldsymbol{x} \geq_{\text{lex}} \boldsymbol{y}$ holds for vectors $\boldsymbol{x}, \boldsymbol{y} \in \mathbb{R}^n$ if $\boldsymbol{x} = \boldsymbol{y}$ or, for the first position i at which $\boldsymbol{x}$ and $\boldsymbol{y}$ differ, we have $x_i > y_i$. Given an automorphism $\gamma \in \Gamma$, we enforce $\boldsymbol{x} \geq_{\text{lex}} \gamma(\boldsymbol{x})$ by the lexicographic reduction algorithm (van Doornmalen & Hojny 2025).

The second approach defines for every node of the branch-and-bound tree a possibly different partial lexicographic order. To define the order at a node b of the branch-and-bound tree, let $x_{i_1}, \dots, x_{i_j}$ be the ordered sequence of branching variables from the root node to node b. Two vectors $\boldsymbol{x}, \boldsymbol{y} \in \mathbb{R}^n$ then satisfy the partial lexicographic order $\geq_{\text{lex},b}$ at node b if $(x_{i_1}, \dots, x_{i_j}) \geq_{\text{lex}} (y_{i_1}, \dots, y_{i_j})$. Ostrowski (2009) has shown that a valid symmetry handling approach is to enforce $\geq_{\text{lex},b}$ at every node b of the branch-and-bound tree. We enforce this by the lexicographic reduction algorithm and the orbital fixing method (Margot 2003, Ostrowski et al. 2011), which are both shown to be compatible by van Doornmalen & Hojny (2025).

Preliminary experiments have shown that the second approach yields better reductions of running time. For this reason, we will only discuss the impact of the second approach in our computational study.

3.3 Further algorithmic enhancements

Two additional components are used to enhance the performance of our branch-and-price algorithm. The first is the *connectivity cut*, a valid inequality that strengthens the LP relaxation of $\text{ILP}^{\star}_{\text{E2}}$ by enforcing a lower bound on the total vertex cut cost derived from the weighted vertex connectivity of the graph; full details are provided in Appendix C. The second is the *Iterative Disconnection Heuristic* (IDH), a constructive algorithm that produces a high-quality initial feasible solution for the k-VCP, enabling early pruning of the BP tree; the implementation details are given in Appendix D.

4 Computational experience

Our computational study pursues three main objectives. The first goal is to assess the impact of the main algorithmic enhancements introduced in the new BP algorithm based on $\text{ILP}^{\star}_{\text{E2}}$, described in Section 3, denoted by BP*, and implemented in `SCIP`, a framework and solver for

ILP models that natively supports all elements of a full branch-and-price algorithm. The second is to compare the performance of BP* with that of the BP algorithm based on ILP_{E1} proposed by Cornaz et al. (2019) and denoted by BP†. The third goal is to benchmark BP* against state-of-the-art exact approaches from the literature, namely those based on ILP_{C} and ILP_{N}, both tackled using `CPLEX`, a commercial solver for ILP models. Throughout this section, COMP denotes the approach that uses `CPLEX` to solve ILP_{C}, strengthened by a symmetry-breaking preprocessing as proposed in Cornaz et al. (2019). We denote by HYB the BC algorithm based on an enhanced version of ILP_{N}, implemented within `CPLEX`, which natively supports branch-and-cut. This model is further strengthened by incorporating a coefficient downlifting procedure for Constraints (3b) together with additional valid inequalities, as proposed by Furini et al. (2020).

All experiments are conducted on the benchmark instances for the k-VCP introduced by Furini et al. (2020). The benchmark contains 304 instances, each considered in two variants: an unweighted version, where all vertices have unit cost, and a weighted version obtained by assigning integer vertex costs in the interval $[1, 10]$. The instances cover values of $k \in \{5, 10, 15, 20\}$. A detailed description of the full benchmark set of instances is provided in Appendix E.

All experiments comparing BP* against BP†, HYB, and COMP were executed single-threaded with a time limit of one hour on the compute cluster of the Chair of Operations Research at RWTH Aachen University, running Debian 11. The implementations of HYB and COMP are publicly available at `https://github.com/paoloparonuzzi/k-Vertex-Cut-Problem`, while the code for BP† was obtained through personal communication with the authors. The cluster nodes are equipped with Intel Xeon L5630 quad-core processors operating at 2.13 GHz and 16 GB of DDR3 RAM.

The remainder of this section is organized as follows. Section 4.1 describes the computational setup and the implementation details of BP*. Section 4.2 presents an ablation study aimed at quantifying the contribution of each algorithmic component to the overall performance of BP* and at comparing BP* with BP†. In Section 4.4 we report a detailed performance comparison between BP* and the other state-of-the-art exact approaches, namely COMP and HYB. Finally, in Section 4.5, we compare the strength of the LP relaxations of the different ILP models.

4.1 Computational setup and implementation details of BP*

BP* has been implemented in `C++` using the BP framework `SCIP` 9.2.3 (Bolusani et al. 2024), which we extended through custom plugins to support the tailored column generation scheme. LP relaxations within `SCIP` are solved using the LP solver `SoPlex` 7.1.5. This choice is motivated by its native integration within the `SCIP` optimization suite and its availability, which aligns with our goal of developing a fully open-source algorithm. Moreover, preliminary tests on the benchmark set of unweighted instances indicated that using `CPLEX` as an LP solver did not yield substantial performance gains: when using `SoPlex`, we were indeed able to solve one additional instance within a one-hour time limit, compared to `CPLEX`. Maximum flow problems arising in the pricing subproblem and in the initial IDH are solved using the `LEMON` graph library (Lemon 2014). The complete `C++` source code, together with all benchmark instances and detailed computational results, is publicly available at `https://doi.org/10.5281/zenodo.17897865`.

This implementation is intended to facilitate reproducibility and to serve as a reference platform for the development and fair comparison of future exact algorithms for the k-VCP and related problems.

4.2 Evaluation of components of BP* and comparison with BP†

Extensive preliminary experiments showed that the construction of the edge-covering family of cliques used in Constraints (10d) has a significant impact on the performance of BP*. For this reason, in addition to the constructive algorithm described in Section 3.1.1, we test two alternative strategies. The first alternative uses a modified version of the constructive algorithm in which, during the construction of a clique, a vertex is added only if it is connected to all vertices currently in the clique by uncovered edges. This modification produces an *edge-partitioning* family of cliques, meaning that each edge belongs to exactly one clique. We denote the corresponding variant of BP* by BP$^\star_a$. The second alternative uses the edge set itself as an edge-partitioning family of cliques. The corresponding variant of BP* is denoted by BP$^\star_b$.

A second key factor in the performance of BP* is the relaxation of Constraints (4b) and (4c) from equalities to inequalities, as in (10b) and (10c). We therefore test a third variant of BP*, denoted as BP$^\star_c$, which uses the equality constraints.

Figure 4 shows performance profiles (see Dolan & Moré 2002) comparing BP*, its three variants, and BP† on the 304 unweighted instances, with a time limit of one hour. For a fair comparison, we re-ran BP† on our hardware using a recent version of `CPLEX` (22.1.0) as LP solver, obtaining results consistent with those reported in Furini et al. (2020). Indeed, Furini et al. (2020) report only three more instances solved within the time limit: this gap can be explained by the significantly higher computational power of the hardware used in their experiments. The solid red curve represents BP*, the three dash-dotted curves correspond to the BP* variants introduced above, and the dotted blue curve represents BP†.

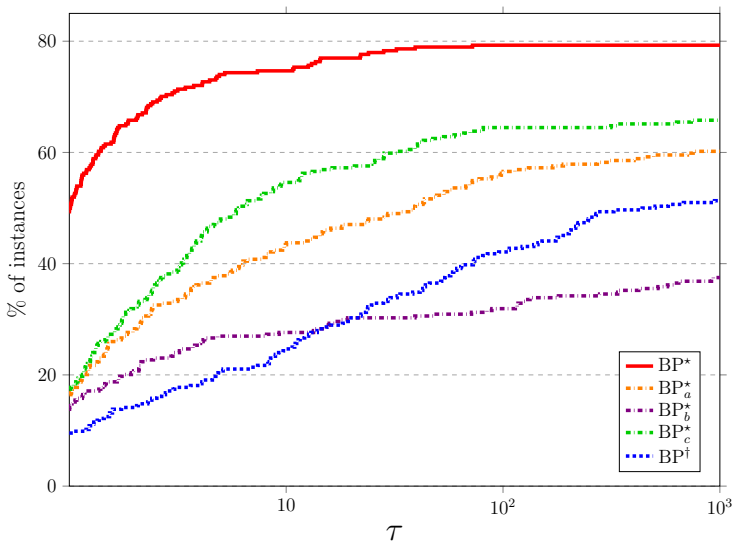


Figure 4: Performance profiles comparing BP* with its three variant and BP† on the benchmark set of unweighted instances, with a time limit of one hour.

Figure 4 highlights the strong impact of the construction of the clique family $\mathscr{C}'$ on the

performance of BP*. In particular, both BP$^\star_a$ and BP$^\star_b$ are clearly dominated by BP*: moving from an edge-covering family of cliques to an edge-partitioning one, and then to single edges, leads to a progressive deterioration of performance, confirming that the construction of $\mathscr{C}'$ is a key component of the algorithm. This is also reflected in the number of instances solved within the time limit: BP* solves 242 instances, compared with 189 for BP$^\star_a$ and only 122 for BP$^\star_b$. A similar effect is observed when comparing BP* with BP$^\star_c$. The performance profile of BP$^\star_c$ grows substantially more slowly and plateaus at a lower fraction of solved instances, with 205 instances solved versus 242 for BP*. This confirms that using Constraints (10b) and (10c) yields a significant performance gain. Finally, we note that the implementation of BP† uses an edge-partitioning family of cliques for Constraints (2d), as in BP$^\star_a$. BP† solves 165 instances and is therefore substantially outperformed by BP* and all its variants except BP$^\star_b$. Interestingly, the comparison between BP† and BP$^\star_a$ shows that, even when using the same type of clique family, the new algorithm achieves a marked improvement.

In the following, we highlight the main differences between BP*, based on ILP$^\star_{\text{E2}}$, and BP†, based on ILP$_{\text{E1}}$, and discuss the reasons we think are behind their performance gap. We first observe that the construction of the clique family $\mathscr{C}'$ has a significant impact on the strength of the LP relaxation of both ILP formulations (see also Section 4.5). In particular, the edge-covering family of cliques used in BP* induces stronger constraints than the edge-partitioning family adopted in BP†. Moreover, on the unweighted benchmark instances, constructing an edge-partitioning family of cliques produces 392 cliques on average, whereas the edge-covering construction used in BP* produces 194 cliques on average. Hence, the latter not only yields stronger constraints, but also does so with a smaller initial family, which further contributes to the improved performance of BP*.

Another important difference concerns Constraints (2b) and (10b). BP† enforces the equality constraint (2b), which requires selecting exactly k clusters, whereas BP* uses the inequality constraint (10b), which only requires selecting at least k clusters. This difference can significantly affect the column generation process. In particular, the inequality constraint does not force the algorithm to generate columns corresponding to clusters that merge multiple connected components, if these are associated with already existing columns. For instance, suppose that columns associated with two clusters, each containing a single connected component, have already been generated: if an optimal solution of ILP$_{\text{E1}}$ requires a single cluster obtained by merging these two components, BP† must continue the column generation process until the corresponding merged cluster is produced. In contrast, BP* may terminate earlier, since the two connected components can be selected separately and still satisfy Constraint (10b).

The column generation process of the two algorithms is also affected by an additional element of difference between them. BP† enforces Constraints (2c), which require that each vertex belongs to at most one cluster in any feasible solution of ILP$_{\text{E1}}$. As introduced in Section 1.1.2, BP† adopts a bilevel branching scheme: at the first level, a partially covered vertex $v \in \mathcal{V}$ (i.e., one for which $\sum_{S \in \mathscr{S}(v)} \lambda_S$ is fractional) is selected, and two child nodes are created by forcing v either to belong to the cut ($\sum_{S \in \mathscr{S}(v)} \lambda_S = 0$) or to a cluster ($\sum_{S \in \mathscr{S}(v)} \lambda_S = 1$). Such branching decisions may require the generation of new columns in order to restore feasibility of the RMP. In contrast, BP* uses Constraints (10c), which impose that a vertex not assigned to

the cut must be covered by a selected cluster, and vice versa. These constraints are formulated as inequalities, allowing a (fractional) LP solution to simultaneously assign a vertex to a cluster and to the cut. This is particularly advantageous during column generation. For instance, when branching forces a vertex into the cut, BP* does not necessarily require the generation of new columns associated with clusters that explicitly exclude that vertex: indeed, the restricted master problem can remain feasible by reusing existing cluster columns.

Furthermore, as detailed in Section 1.1.2, first-level branching on individual vertices in BP† is not sufficient to enforce integrality. As a consequence, BP† applies a second branching level based on a Ryan–Foster-type scheme on pairs of vertices: when two vertices $u, v \in \mathcal{V}$ are partially assigned to the same cluster, two child nodes are generated by enforcing that u and v must either belong to the same cluster or to different clusters. This additional branching level may significantly enlarge the search tree, as fractional solutions can persist even when all vertices are integrally assigned. Moreover, it complicates the pricing subproblem, since newly generated columns must respect pairwise compatibility decisions. In contrast, BP* does not require any additional branching on vertex pairs, leading to a smaller search tree and keeping the pricing problem unchanged.

Finally, on the implementation side, the code by Cornaz et al. (2019) uses a custom branch-and-bound framework to implement BP†, where the branching tree is explored in a depth-first fashion, and first-level branching decisions are based on the lowest-index vertex v for which $\sum_{S \in \mathscr{S}(v)} \lambda_S \notin \mathbb{Z}$ in an LP solution. On the other hand, BP* makes use of the branch-and-bound framework of `SCIP`. Next to more sophisticated tree exploration and branching rules, `SCIP` also implements advanced methods such as strong branching (Applegate et al. 1995) which help to reduce the size of the branching tree. Moreover, the implementation of BP* benefits from the primal heuristics that are available in `SCIP`, whereas the implementation of BP† requires LP solutions to be integral to improve the primal bound.

4.2.1 Evaluation of additional components of BP*

We also evaluated the impact of three additional features: the symmetry breaking techniques detailed in Section 3.2, providing BP* with the initial primal solution returned by IDH (see Section 3.3) and adding the connectivity cut (13) to ILP$^\star_{\text{E2}}$. Symmetry handling on the $\boldsymbol{x}$-variables has only a modest visible impact on the running time for most instances, since the corresponding branching trees are relatively small and the techniques detailed in Section 3.2 are particularly effective on large search trees. Nonetheless, their inclusion allows BP* to solve four additional instances within the time limit compared with the variant of the algorithm without symmetry handling, which proves useful in terms of the overall number of instances solved and motivates keeping these techniques in the final version of the algorithm. Providing BP* with the initial solution returned by IDH improves the performance of the algorithm as well: with an initial primal solution, BP* solves eight more instances within the time limit than the variant started without any initial solution. Similarly, the connectivity cut is generally beneficial and is therefore included in our default configuration. Enabling it for moderate values of k (≤ 15), as discussed in Section 3.3, allows BP* to solve three additional instances within the time limit compared with the variant without it.

4.3 Quantitative numerical results

Table 1 summarizes the behavior of the BP* configuration on the unweighted benchmark, grouped by value of k. For each block of instances, we report the total number of instances and that of the optimally solved ones (#inst, #opt), the average size of the initial clique family and the average size of the cliques in terms of number of vertices (#clq, #vert), the average solution time and the portion of time spent in pricing (time total, pricing) on the optimally solved instances, the average number of generated columns (total, at the root node, and in Farkas pricing iterations) for the closed instances, the average number of explored branch-and-bound nodes and tree depth (B&B nodes, depth) in the optimally solved instances, and finally the average integrality gap on the subset of instances whose optimal value is known and for which our best configuration algorithm manages to compute a valid dual bound at the root node within the time limit. More precisely, let $z^\star$ denote the optimal value of a given instance, whenever it is known either from our runs or from the literature (see Furini et al. 2020), and let z^{LP} be the value of the root-node LP relaxation produced by BP*. For each such instance on which BP* computes a valid dual bound z^{LP} within the time limit, we define the *integrality gap* as $100 \cdot \frac{z^\star - z^{\mathrm{LP}}}{z^\star}$, which is therefore expressed as a percentage.

Overall, BP* solves 242 out of 304 unweighted instances, i.e., roughly 80% of the test set, with fairly stable performance across different values of k. The size of the initial edge-covering family of cliques remains moderate (about 195 cliques on average, with a mean of around 7 vertices per clique), with an increase in the number of clique—and an associated decrease in the number of vertices per clique—as the value of k increases. Average total running times for solved instances are on the order of one to two minutes (104.3 seconds on average), with pricing accounting for about one third of the time, which confirms that column generation is not a bottleneck component from the computational perspective. The number of generated columns is in the low thousands (about 2,760 on average), with a substantial fraction already produced at the root node and a relatively small number of Farkas columns. The branch-and-bound tree remains shallow and compact on the solved instances: on average, fewer than 50 nodes are explored, with depth around 5, which is consistent with a strong LP relaxation and effective branching. Finally, the average integrality gaps are very small: on instances whose optimal value is known and for which BP* computes a valid root-node dual bound within the time limit, the average difference between this LP bound and the optimum is about 11%. This indicates that the LP relaxation of our formulation is quite strong and that often our branch-and-price starts from a high-quality bound.

4.4 Computational comparison between exact approaches

In this section, we compare the three exact approaches COMP, HYB, and BP* on the benchmark set, considering both unweighted and weighted instances and a time limit of 3600 seconds. To ensure a fair comparison, we re-executed COMP and HYB on our hardware using the same time limit and a more recent version of `CPLEX` (22.1.0). Overall, the relative performance trends of COMP and HYB are in line with those reported in Furini et al. (2020). Nevertheless, since the computational platform used in Furini et al. (2020) was more powerful than ours, some of the hardest instances are not closed within the time limit in our experiments. As a result, we

Table 1: Summary statistics for the final BP* configuration on the benchmark set of unweighted instances, grouped by value of k, with a time limit of one hour.

			cliques		time		gen cols			branching tree		
	#inst	#opt	#clq	#vert	total	pricing	total	root	farkas	nodes	depth	gap (%)
$k = 5$	107	85	176.3	8.1	98.6	18.3	2,395.3	1,124.6	57.4	56.1	4.4	14.0
$k = 10$	80	63	195.7	6.4	160.4	72.1	3,462.9	843.4	186.7	72.3	6.6	11.5
$k = 15$	65	50	201.5	5.5	48.9	12.7	2,124.9	1,200.1	5.4	10.2	3.2	7.1
$k = 20$	52	44	226.9	5.3	98.0	57.7	3,198.7	894.8	410.1	30.4	6.4	10.2
Tot/Avg	304	242	195.4	6.6	104.3	38.3	2,763.4	1,025.2	144.4	46.1	5.1	11.2

solve fewer instances than what reported in Furini et al. (2020): on the unweighted set, we solve 7 and 3 fewer instances with HYB and COMP, respectively, while on the weighted set both methods solve 7 fewer instances.

Table 2 reports, for each $k \in \{5, 10, 15, 20\}$, the number of instances solved to proven optimality by each algorithm. On unweighted instances, BP* solves 242 out of 304 cases, against 195 for HYB and 165 for COMP. The advantage of BP* over COMP increases with k: for $k = 5$ the two methods are relatively close whereas, for $k = 20$, BP* solves more than twice as many instances as the other algorithm. Conversely, the performance gap between BP* and HYB decreases as k grows: although BP* keeps solving almost the same fraction of instances for all values of k, the performance of HYB improves with $k = 15$ and $k = 20$, thus reducing the gap between the two algorithms from 18 instances ($k = 5$) to just 6 instances ($k \in \{15, 20\}$). The results are similar on weighted instances, where BP* solves 239 instances, while HYB and COMP solve 201 and 170 instances, respectively. For $k = 5$, COMP is slightly ahead of the other two algorithms, confirming its effectiveness on instances with a smaller value of k, but for $k \geq 10$ BP* obtains the largest number of solved instances in all cases. Overall, these results indicate that BP* dominates HYB and COMP in terms of number of closed instances on both test sets, and across almost all values of k.

Table 2: Performance comparison of the exact algorithms COMP, HYB and BP* on the full set of weighted and unweighted instances with a time limit of one hour.

		Unweighted instances			**Weighted instances**		
	#inst	BP* #opt	HYB #opt	COMP #opt	BP* #opt	HYB #opt	COMP #opt
$k = 5$	107	85	67	**88**	85	68	**90**
$k = 10$	80	**63**	46	32	**61**	49	34
$k = 15$	65	**50**	44	28	**50**	45	28
$k = 20$	52	**44**	38	17	**43**	39	18
Tot	304	**242**	195	165	**239**	201	170

The trends observed in Table 2 are confirmed by the plots in Figure 5, showing the cumulative percentage of instances solved to optimality over time by each of the three algorithms. For both unweighted and weighted instances, the curve of BP* lies above those of HYB and COMP over almost the entire time horizon. In the weighted case, HYB and BP* exhibit very similar performance for running times up to about one second, but afterwards the curve of BP* grows more rapidly and remains clearly ahead. This shows that BP* is not only competitive on the easiest instances, but also both faster and more effective on the more difficult ones, namely those requiring tens or hundreds of seconds to be solved to optimality.

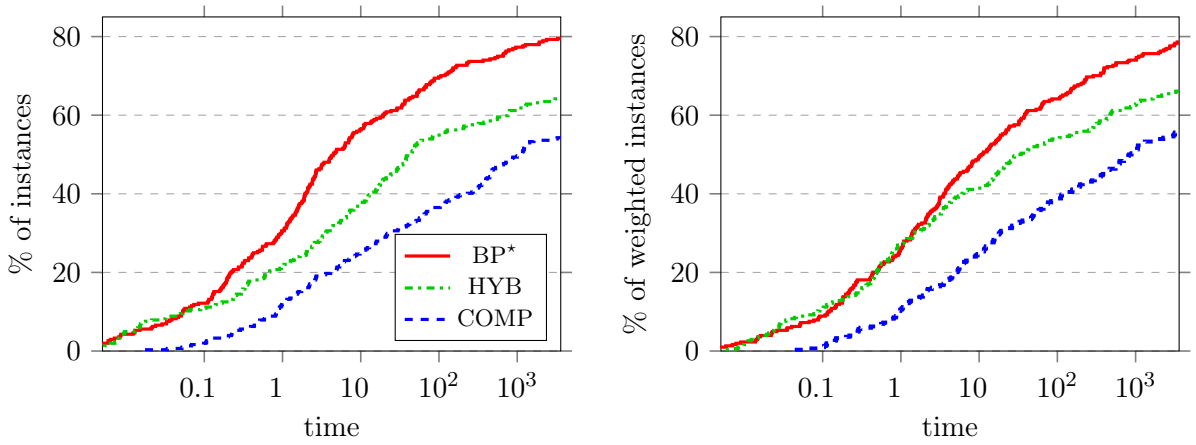


Figure 5: Cumulative percentage of instances solved to optimality over time by each algorithm on the complete sets of 304 unweighted instances (left) and 304 weighted instances (right), with a time limit of one hour.

To further assess the contribution of BP* relative to previously available exact approaches, we compare our results with the instance-wise outcomes reported in Furini et al. (2020), obtained under the same time limit but on a substantially more powerful hardware. In that study, the computational campaign involved three methods, namely COMP, HYB, and BP†. An instance is considered previously solved if it was solved to proven optimality by at least one of these approaches in Furini et al. (2020). With respect to this baseline, BP* closes 73 additional instances that were not solved by any exact method in the reference study: 33 unweighted and 40 weighted instances. For reference, among the three algorithms tested in Furini et al. (2020), HYB closes 33 instances not solved by either of the other methods (17 unweighted and 16 weighted), COMP closes 8 (2 unweighted and 6 weighted), and BP† closes 3 (1 unweighted and 2 weighted). Finally, some instances that were solved in Furini et al. (2020) are not closed by BP* within the time limit in our experiments. Detailed instance-wise results, including optimal or best known objective values, are reported in Appendix G, where we highlight both the instances solved for the first time by BP* and those on which BP* fails while at least one method from Furini et al. (2020) succeeds.

4.5 Strength of the LP relaxations

In this section, we compare the strength and computational cost of the LP relaxations associated with the different formulations considered in this paper. The goal of this analysis is

twofold: on the one hand, to quantify how the choice of the initial clique family in $\text{ILP}^{\star}_{\text{E2}}$ (edge-covering family of cliques, edge-partitioning family of cliques, or single edges) affects the quality of the root-node bound; on the other hand, to position the resulting relaxations with respect to previously proposed models, namely the enhanced version of ILP_{N} introduced in Section 4 and detailed in Furini et al. (2020) (denoted as ILP_{H}), the natural formulation ILP_{N}, and the compact formulation ILP_{C} with the symmetry breaking enhancement mentioned in Section 4 (denoted as $\text{ILP}^{\star}_{\text{C}}$).

Table 3 reports, for each value of k, the average integrality gap between the LP bound and the optimal integer solution value (columns "gap") and the average solution time of the root-node LP (columns "time") for all formulations under comparison, on the subset of instances whose optimal solution value is known and such that it was possible to compute a valid LP bound within one hour of running time. Variants $\text{ILP}^{\star}_{\text{E2}}$, $\text{ILP}^{\star}_{\text{E2, a}}$, and $\text{ILP}^{\star}_{\text{E2, b}}$ correspond to the three choices of initial clique family $\mathscr{C}'$: $\text{ILP}^{\star}_{\text{E2}}$ uses the edge-covering family of cliques described in Section 3.1, $\text{ILP}^{\star}_{\text{E2, a}}$ uses the edge-partitioning family of cliques of Cornaz et al. (2019), and $\text{ILP}^{\star}_{\text{E2, b}}$ replaces cliques by single edges. None of these variants includes the connectivity cut introduced in Section 3.3, so as to isolate the effect of the initial clique family on the strength of the relaxation.

Among all variants, $\text{ILP}^{\star}_{\text{E2}}$ consistently has the strongest relaxation: its average gaps range from 15.6% for $k = 5$ down to 6.4% for $k = 20$, with an overall average of 11.1%. The two variants based on weaker initial clique families yield progressively weaker bounds: $\text{ILP}^{\star}_{\text{E2, a}}$ has gaps between 22.9% and 8.3%, while $\text{ILP}^{\star}_{\text{E2, b}}$ is much weaker, with gaps between 52.3% and 23.6%, comparable to those of the natural formulation. This shows that the initial clique family is a decisive modeling aspect: moving from an edge-covering family of cliques to an edge-partitioning family of cliques, and further to single edges, quickly weakens the LP relaxation down to the level of the natural model.

As a computational counterpart to the theoretical results in Section 2.2, we observe that $\text{ILP}^{\star}_{\text{E2, b}}$ and ILP_{N} provide the same LP bound (identical gap entries for each k); nonetheless, $\text{ILP}^{\star}_{\text{E2, b}}$ requires substantially more time to compute it, especially for $k = 5$ and $k = 10$. In terms of relaxation strength alone, $\text{ILP}^{\star}_{\text{E2, b}}$ therefore does not improve over the natural formulation, while incurring a higher computational overhead. Such overhead can be explained by the high number of columns generated at the root node (roughly 50% more than $\text{ILP}^{\star}_{\text{E2}}$), but also by the increased complexity of the pricing problem, which takes more time to be solved due to the greater number of cliques considered.

The hybrid model ILP_{H} yields intermediate gaps, systematically better than ILP_{N} but clearly looser than $\text{ILP}^{\star}_{\text{E2}}$, with average gaps around 34% versus 11.1% for $\text{ILP}^{\star}_{\text{E2}}$. Finally, the compact formulation $\text{ILP}^{\star}_{\text{C}}$ has by far the weakest relaxation, with gaps close to 90% across all values of k, although with very small LP times.

Overall, the table illustrates that the edge-covering family of cliques used in $\text{ILP}^{\star}_{\text{E2}}$ is key to obtaining a much stronger LP bound than all other formulations considered. Although computing the LP relaxation of $\text{ILP}^{\star}_{\text{E2}}$ requires more root-node time than ILP_{H}, ILP_{N}, or $\text{ILP}^{\star}_{\text{C}}$, its LPs remain reasonably fast to solve and the improvement in bound quality is substantial. This tighter relaxation of $\text{ILP}^{\star}_{\text{E2}}$ is a promising indication of the effectiveness of $\text{BP}^{\star}$ and is

likely to translate, in the full branch-and-price algorithm, into fewer explored nodes and a larger number of instances closed within the time limit compared with the other existing approaches.

Table 3: Average integrality gaps (in %) and root-node solution times (in seconds) for all formulations on the unweighted benchmark set, grouped by k

		$\mathbf{ILP^{\star}_{E2}}$		$\mathbf{ILP^{\star}_{E2,\,a}}$		$\mathbf{ILP^{\star}_{E2,\,b}}$		$\mathbf{ILP_{H}}$		$\mathbf{ILP_{N}}$		$\mathbf{ILP^{\star}_{C}}$	
k	#inst	gap	time	gap	time	gap	time	gap	time	gap	time	gap	time
5	88	**15.6**	59.3	22.9	119.8	52.3	193.9	39.6	17.0	52.3	1.9	90.6	0.2
10	69	**10.9**	61.8	16.5	196.1	40.4	149.4	37.2	29.6	40.4	2.8	92.2	1.9
15	52	**7.5**	31.6	10.8	90.0	26.9	72.6	26.5	13.6	26.9	3.8	88.5	7.6
20	42	**6.4**	48.2	8.3	120.0	23.6	98.7	23.2	13.0	23.6	5.3	90.3	10.6
Tot/Avg	251	**11.1**	52.4	16.2	134.6	38.9	140.6	33.5	19.1	38.9	3.1	90.5	4.0

5 Conclusions

Despite its practical relevance for numerous areas such as network design or vulnerability assessment, existing methods for exactly solving the k-VCP are not powerful enough to solve medium sized instances. This article therefore took a fresh look into the problem and identified unused potential in an existing branch-and-price framework: while all variables in the previous extended formulation were required to take binary values, our in-depth study revealed that the integrality requirement on those variables can be dropped when introducing the binary variables of the natural formulation. This new perspective on the extended model allowed us to significantly simplify the underlying branching scheme and to enhance the new branch-and-price algorithm by novel components. While results by Furini et al. (2020) suggest that branch-and-price was not the right technique for tackling the k-VCP, we demonstrate in an extensive numerical study that, when properly set up, the new branch-and-price substantially improves on the state of the art. Our algorithmic framework thus not only provides a new benchmark for solving the k-VCP to proven optimality, but also changes the perspective on how to solve the k-VCP: branch-and-price strikes back by solving for the first time 73 open instances.

Moreover, although our algorithm can be considered state of the art, there are still instances in our test set that remain unsolved. This suggests several directions for further improvement. One promising line of research is the development of more powerful presolving routines aimed at simplifying particularly difficult instances. Another potential enhancement is the incorporation of connectivity constraints within the subsets associated with the variables of the extended formulation. Considering such constraints could further strengthen the LP relaxation, but this would also have to be taken into account in the pricing problem, which may significantly increase its complexity. A careful study of this trade-off between bound quality and pricing difficulty can therefore be an interesting topic for future work.

The theoretical insights developed in this work may indicate a more general pathway for deriving extended formulations from natural bilevel models (see, e.g., Furini et al. (2020, 2022)). Such models arise frequently in graph partitioning problems, where a leader removes vertices or edges and a follower solves a subproblem that captures a structural property of the modified graph. As discussed in Section 2.1.1, if the follower's problem admits a polyhedral description with a totally unimodular constraint matrix, then dualization and subsequent linearization yield an extended formulation of the original problem with an exponential number of variables which can then be solved by means of a branch-and-price algorithm. For the k-VCP, this procedure led to formulation ILP_{E2}, in other settings the same construction may give rise to new formulations and, in turn, potentially efficient branch-and-price algorithms. Among others, this concerns the k-way vertex cut problem (Berger et al. 2014), whose optimization version aims at maximizing the number of connected components of the residual graph under a budget on the cardinality of the vertex cut. We believe that our branch-and-price algorithm can be easily adapted and used to tackle this problem as well. More generally, the same approach might apply to other critical vertex detection problems that seek to fragment the graph by deleting vertices.

References

Achterberg, T. (2007). *Constraint integer programming* (Doctoral dissertation). TU Berlin.

Applegate, D., Bixby, R., Chvatal, V., & Cook, B. (1995). *Finding cuts in the TSP (a preliminary report)* (Tech. Rep.).

Barahona, F., & Jensen, D. (1998). Plant location with minimum inventory. *Mathematical Programming*, *83*(1), 101–111.

Barnhart, C., Johnson, E. L., Nemhauser, G. L., Savelsbergh, M. W. P., & Vance, P. H. (1998). Branch-and-price: Column generation for solving huge integer programs. *Operations Research*, *46*(3), 316–329.

Ben-Ameur, W., & Didi Biha, M. (2012). On the minimum cut separator problem. *Networks*, *59*(1), 30–36.

Berger, A., Grigoriev, A., & v. d. Zwaan, R. (2014). Complexity and approximability of the k-way vertex cut. *Networks*, *63(2)*, 170–178.

Bolusani, S., Besançon, M., Bestuzheva, K., Chmiela, A., Dionísio, J., Donkiewicz, T., ... Xu, L. (2024). *The SCIP optimization suite 9.0.*

Borndörfer, R., Ferreira, C., & Martin, A. (1998). Decomposing matrices into blocks. *SIAM Journal on Optimization*, *9*(1), 236-269.

Cornaz, D., Furini, F., Lacroix, M., Malaguti, E., Mahjoub, A. R., & Martin, S. (2019). The vertex k-cut problem. *Discrete Optimization*, *31*, 8–28.

Desaulniers, G., Desrosiers, J., & Solomon, M. M. (Eds.). (2005). *Column generation.* Boston, MA: Springer.

de Souza, C., & Balas, E. (2005). The vertex separator problem: algorithms and computations. *Mathematical Programming*, *103(3)*, 609-631.

Desrosiers, J., Lübbecke, M., Desaulniers, G., & Gauthier, J. B. (2026). *Branch-and-price.* Springer, Cham.

Dolan, E. D., & Moré, J. J. (2002). Benchmarking optimization software with performance profiles. *Mathematical Programming*, *91*(2), 201–213.

Edmonds, J. (1970). Submodular functions, matroids and certain polyhedra. In R. Guy, H. Hanani, N. Sauer, & J. Schönheim (Eds.), *Combinatorial structures and their applications* (pp. 69–87). Gordon and Break, New York.

Furini, F., Ljubić, I., Malaguti, E., & Paronuzzi, P. (2020). On integer and bilevel formulations for the k-vertex cut problem. *Mathematical Programming Computation*, *12*, 133–164.

Furini, F., Ljubic, I., Malaguti, E., & Paronuzzi, P. (2022). Casting light on the hidden bilevel combinatorial structure of the capacitated vertex separator problem. *Operations Research*, *70*(4), 2399–2420.

Furini, F., Ljubić, I., San Segundo, P., & Martin, S. (2019). The maximum clique interdiction problem. *European Journal of Operational Research*, *277*(1), 112–127.

Furini, F., Ljubić, I., San Segundo, P., & Zhao, Y. (2021). A branch-and-cut algorithm for the edge interdiction clique problem. *European Journal of Operational Research*, *294*(1), 54–69.

Grötschel, M., Lovász, L., & Schrijver, A. (1981). The ellipsoid method and its consequences in combinatorial optimization. *Combinatorica*, *1*(2), 169–197.

Korte, B., & Vygen, J. (2018). *Combinatorial optimization* (6th ed.) (No. 21). Springer, Berlin.

Lalou, M., Tahraoui, M. A., & Kheddouci, H. (2018). The critical node detection problem in networks: A survey. *Computer Science Review*, *28*, 92 - 117.

Lemon graph library. (2014). Available at https://github.com/tpet/lemon.

Lübbecke, M. E., & Desrosiers, J. (2005). Selected topics in column generation. *Operations Research*, *53*(6), 1007–1023.

Margot, F. (2003). Exploiting orbits in symmetric ILP. *Mathematical Programming*, *98*(1–3), 3–21.

Marx, D. (2006). Parameterized graph separation problems. *Theoretical Computer Science*, *351*(3), 394-406.

Ostrowski, J. (2009). *Symmetry in integer programming* (PhD dissertation). Lehigh University.

Ostrowski, J., Linderoth, J., Rossi, F., & Smriglio, S. (2011). Orbital branching. *Mathematical Programming*, *126*(1), 147–178.

Pfetsch, M. E., & Rehn, T. (2019). A computational comparison of symmetry handling methods for mixed integer programs. *Mathematical Programming Computation*, *11*(1), 37–93.

Ryan, D., & Foster, B. (1981). *An integer programming approach to scheduling.* Wren A, ed. Computer Scheduling of Public Transport: Urban Passenger Vehicle and Crew Scheduling (North-Holland, Amsterdam).

Uchoa, E., Pessoa, A., & Moreno, L. (2024). *Optimizing with Column Generation: Advanced branch-cut-and-price algorithms (Part I)* (Tech. Rep. No. L-2024-3). Cadernos do LOGIS-UFF, Universidade Federal Fluminense, Engenharia de Produção.

van Doornmalen, J., & Hojny, C. (2025). A unified framework for symmetry handling. *Mathematical Programming*, *212*, 217–271.

Zachary, W. W. (1977). An information flow model for conflict and fission in small groups. *Journal of anthropological research*, *33*(4), 452–473.

A Deferred proofs

Proof. Proof of Lemma 2 Let $\boldsymbol{x} \in \{0,1\}^{\mathcal{V}}$ and let $\tilde{\mathcal{G}} \coloneqq \mathcal{G}(\boldsymbol{x}) = (\tilde{\mathcal{V}}, \tilde{\mathcal{E}})$ as defined above. Let $\mathscr{M} \subseteq 2^{\tilde{\mathcal{E}}}$ be the collection of subsets of $\tilde{\mathcal{E}}$ that define an acyclic subgraph of $\tilde{\mathcal{G}}$. The set $\mathscr{M}$ is the so-called cycle matroid of $\tilde{\mathcal{G}}$, see Korte & Vygen (2018), with associated rank function $r\colon 2^{\tilde{\mathcal{E}}} \to \mathbb{Z}_+$,

$$S \mapsto r(S) \coloneqq |S| - \text{number of connected components of } \tilde{\mathcal{G}}[S].$$

Due to Edmonds (1970), the polytope

$$P = \{y \in \mathbb{R}_+^{\tilde{\mathcal{E}}} : y(S) \leq r(S) \text{ for all } S \subseteq \tilde{\mathcal{E}}\}$$

is integral and its vertices are the incidence vectors of (edge sets of) acyclic subgraphs of $\tilde{\mathcal{E}}$. Moreover, it is easy to see that if $S \subseteq \tilde{\mathcal{E}}$ has multiple connected components, then the inequality $y(S) \leq r(S)$ can be dominated by the sum of the inequalities for the respective connected components. Therefore,

$$P = \{y \in \mathbb{R}_+^{\tilde{\mathcal{E}}} : y(S) \leq |S|-1 \text{ for all } S \subseteq \tilde{\mathcal{E}}\} = \{y \in \mathbb{R}_+^{\tilde{\mathcal{E}}} : y(S) \leq |S|-1 \text{ for all } S \subseteq \tilde{\mathcal{V}},\ |S| \geq 2\}.$$

In particular, by selecting $S = \{u, v\}$, we find $y_e \leq 1$ for all $e \in \tilde{\mathcal{E}}$.

The assertion follows by lifting P from $\mathbb{R}^{\tilde{\mathcal{E}}}$ to $\mathbb{R}^{\mathcal{E}}$. This is achieved by introducing variables y_e for all $e = \{u, v\} \in \mathcal{E} \setminus \tilde{\mathcal{E}}$ and fixing $y_e = 0$ (via the inequalities $y_e \geq 0$, $y_e \leq 1 - x_u$, and $y_e \leq 1 - x_v$). □ □

Proof. Proof of Theorem 3 Let (x', λ', β') be an optimal solution to (8). Then, $x' \in \{0,1\}^{\mathcal{V}}$, and we construct an optimal solution $(\bar{x}, \bar{\lambda}, \bar{\beta})$ with the claimed properties.

Let $\mathscr{K}$ be the set of connected components of the graph arising from $\mathcal{G}$ by removing $\{v \in \mathcal{V} : x'_v = 1\}$. Since x' is a solution to the k-vertex cut problem, we have $|\mathscr{K}| \geq k$. Consider the solution

$$\bar{\boldsymbol{x}} = \boldsymbol{x}', \qquad \bar{\lambda}_S = \begin{cases} 1, & \text{if } S \in \mathscr{K}, \\ 0, & \text{otherwise,} \end{cases} \qquad \bar{\beta}_{ew} = x'_w.$$

We claim that this solution is an optimal solution of (8). Because the objective value is determined by the $\boldsymbol{x}$-variables only and $\bar{\boldsymbol{x}} = \boldsymbol{x}'$, optimality follows once we can show that $(\bar{\boldsymbol{x}}, \bar{\boldsymbol{\lambda}}, \bar{\boldsymbol{\beta}})$ is feasible.

To show that (8b) holds, note that $(1 - \bar{x}_w)\bar{\beta}_{ew} = (1 - \bar{x}_w)\bar{x}_w = 0$ because $\bar{\boldsymbol{x}} \in \{0,1\}^{\mathcal{V}}$. Therefore,

$$\sum_{S \in \mathscr{S}} (|S| - 1)\bar{\lambda}_S + \sum_{e \in \mathcal{E}} \sum_{w \in e} (1 - \bar{x}_w)\bar{\beta}_{ew} = \sum_{S \in \mathscr{S}} (|S| - 1)\bar{\lambda}_S = \sum_{S \in \mathscr{K}} (|S| - 1) \leq n - \sum_{v \in \mathcal{V}} x_v - k,$$

where that last inequality holds, because $\mathscr{K}$ are the (at least) k connected components of the graph obtained from removing $\{v \in \mathcal{V} : x'_v = 1\}$ from $\mathcal{G}$. That is, (8b) holds.

Regarding (8c), let $e = \{u, v\} \in \mathcal{E}$. If $\bar{\beta}_{eu} + \bar{\beta}_{ev} < 1$, then $\bar{\beta}_{eu} + \bar{\beta}_{ev} = \bar{x}_u + \bar{x}_v = 0$, i.e., neither u nor v is deleted from $\mathcal{G}$. Consequently, since u and v are adjacent, there exists a connected component in $\mathscr{K}$ that contains both u and v. Hence, (8c) is satisfied.

Since $(\bar{\boldsymbol{x}}, \bar{\boldsymbol{\lambda}}, \bar{\boldsymbol{\beta}})$ clearly satisfies the bound and integrality constraints of (8), this solution is feasible for (8), which concludes the proof. □ □

Proof. Proof of Corollary 4 On the one hand, if the k-vertex cut problem for $\mathcal{G}$ is infeasible, also (8) is infeasible. Thus, adding further constraints does not change the feasible region. On

the other hand, when the k-vertex cut problem is feasible, Theorem 3 guarantees the existence of an integer optimal solution, where $\lambda_S = 1$ if and only if S corresponds to a connected component of the reduced graph. When this solution has exactly k connected components, it is immediate that this solution satisfies all of the constraints proposed in the assertion. Otherwise, when the solution has more than k connected components, observe that the solution remains feasible when setting $\lambda_{S_1} = \lambda_{S_2} = 0$ and $\lambda_{S_1 \cup S_2} = 1$ for two distinct connected components S_1 and S_2. That is, we can merge some of the $\boldsymbol{\lambda}$-variables to ensure that exactly k of them take value 1. □ □

Proof. Proof of Proposition 12 Let $(\boldsymbol{x}, \boldsymbol{\lambda}) \in \{0,1\}^{\mathcal{V}} \times \{0,1\}^{\mathscr{S}}$ and let γ be an automorphism of $\mathcal{G}$. We need to show that $(\gamma(\boldsymbol{x}), \gamma(\boldsymbol{\lambda}))$ is feasible for $\text{ILP}^{\star}_{\text{E2}}$ if and only if $(\boldsymbol{x}, \boldsymbol{\lambda})$ is feasible, and that both solutions have the same objective value. The latter is an immediate consequence of γ being an automorphism of $\mathcal{G}$, because

$$\sum_{v \in \mathcal{V}} c_v \gamma(\boldsymbol{x})_v = \sum_{v \in \mathcal{V}} c_v x_{\gamma^{-1}(v)} = \sum_{v \in \mathcal{V}} c_{\gamma(v)} x_v = \sum_{v \in \mathcal{V}} c_v x_v.$$

To prove that feasibility is preserved, we discuss the different families of constraints of $\text{ILP}^{\star}_{\text{E2}}$ in turn.

First, consider (10b). Since γ permutes the entries of both the $\boldsymbol{x}$-variables and $\boldsymbol{\lambda}$-variables, Constraint (10b) holds for $(\boldsymbol{x}, \boldsymbol{\lambda})$ if and only if it holds for $(\gamma(\boldsymbol{x}), \gamma(\boldsymbol{\lambda}))$.

Second, regarding (10c), consider a solution $(\boldsymbol{x}, \boldsymbol{\lambda})$. Then, for every $v \in \mathcal{V}$, we have

$$\sum_{S \in \mathscr{S}(v)} \lambda_S + x_v = \sum_{S \in \mathscr{S}(v)} \lambda_{\gamma^{-1}\gamma(S)} + x_{\gamma^{-1}\gamma(v)} = \sum_{S \in \mathscr{S}(v)} \gamma(\boldsymbol{\lambda})_{\gamma(S)} + \gamma(\boldsymbol{x})_{\gamma(v)} = \sum_{S \in \mathscr{S}(\gamma(v))} \gamma(\boldsymbol{\lambda})_S + \gamma(\boldsymbol{x})_{\gamma(v)}.$$

That is, $(\boldsymbol{x}, \boldsymbol{\lambda})$ satisfies (10c) for vertex v if and only if $(\gamma(\boldsymbol{x}), \gamma(\boldsymbol{\lambda}))$ satisfies (10c) for vertex $\gamma(v)$. As a consequence, $(\boldsymbol{x}, \boldsymbol{\lambda})$ adheres to family (10c) if and only if $(\gamma(\boldsymbol{x}), \gamma(\boldsymbol{\lambda}))$ does.

Finally, to show that $(\boldsymbol{x}, \boldsymbol{\lambda})$ satisfies (10d) if and only if $(\gamma(\boldsymbol{x}), \gamma(\boldsymbol{\lambda}))$ satisfies these constraints, we can use the same arguments as for (10c). This concludes the proof. □ □

B Solution analysis of the Zachary's karate club network

The `karate` instance represents the classical Zachary's karate club network (Zachary 1977), a widely used benchmark in community detection and graph partitioning. In this graph, vertices correspond to members of a university karate club, and edges connect pairs of individuals who were observed interacting outside formal activities. A conflict between the instructor (vertex 1) and the club administrator (vertex 34) eventually led to the split of the club into two factions, a structural characteristic that makes this instance particularly suitable for illustrating graph disconnection phenomena. The optimal solutions of the k-VCP on this network exhibit a progressively increasing level of fragmentation as k grows, with the k-vertex cut shifting from a single highly central vertex to a larger set of influential ones. For $k = 3$, an optimal 3-vertex cut consists of the single vertex 1 (the instructor), whose removal is sufficient to disconnect the graph into exactly three components. For $k = 5$, an optimal 5-vertex cut has cardinality 2 and includes vertices 1 and 2. For $k = 10$, an optimal 10-vertex cut has cardinality 4 and includes vertices 1, 3, 33, and 34, whose removal disconnects the network into ten components. A noticeable property of this network is its low density: many vertices have only two or three neighbors. This explains why the size of optimal vertex cuts remains moderate even for larger

values of k, and why the corresponding solutions contain several components that are essentially singletons (one singleton for $k = 3$ and seven for $k = 10$). As expected, the vertices selected in the cuts tend to be those with high degree, which justifies the repeated appearance of vertices 1 and 34 (the instructor and the club administrator) in optimal solutions for larger values of k.

C The connectivity cut

For both the connectivity cut and the IDH (Appendix D), we use the following notation. Given a connected graph $\mathcal{G}[U]$, with $U \subseteq \mathcal{V}$, and a vector of non-negative vertex costs $\boldsymbol{c}_U = (c_v)_{v \in U}$, we define its *weighted vertex connectivity* as the minimum total cost of a vertex set $W \subseteq U$ whose removal disconnects $\mathcal{G}[U]$. Equivalently, W is such that $\mathcal{G}[U \setminus W]$ has more connected components than $\mathcal{G}[U]$. We denote the weighted vertex connectivity by $\tilde{\kappa}(\mathcal{G}[U], \boldsymbol{c}_U)$ and we set it to $+\infty$ for complete graphs since they cannot be disconnected. In particular, if $\mathcal{G}[U]$ is not complete and $c_v = 1$ for all $v \in U$, then $\tilde{\kappa}(\mathcal{G}[U], \boldsymbol{c}_U)$ coincides with the *vertex connectivity* of $\mathcal{G}[U]$. We extend this definition to graphs that are not necessarily connected by taking the minimum weighted vertex connectivity over their connected components. Thus, if $\mathcal{G}[U]$ is disconnected, then $\tilde{\kappa}(\mathcal{G}[U], \boldsymbol{c}_U)$ is defined as the minimum value of $\tilde{\kappa}(\mathcal{G}[Q], \boldsymbol{c}_Q)$ over all connected components $\mathcal{G}[Q]$ of $\mathcal{G}[U]$, with $Q \subseteq U$.

If the number of connected components of $\mathcal{G}$ is smaller than k, as in all non-trivial k-VCP instances, then every feasible integer solution of $\text{ILP}^{\star}_{\text{E2}}$ satisfies

$$\sum_{v \in \mathcal{V}} c_v x_v \geq \tilde{\kappa}(\mathcal{G}, \boldsymbol{c}_{\mathcal{V}}), \tag{13}$$

and inequality (13) can be added to $\text{ILP}^{\star}_{\text{E2}}$ to strengthen its LP relaxation. We refer to (13) as the *connectivity cut*. Intuitively, this inequality states that, whenever the current graph has fewer than k connected components, any feasible solution must remove vertices whose total cost is at least the minimum cost required to disconnect one of its connected components.

For a given $U \subseteq \mathcal{V}$ such that $\mathcal{G}[U]$ is a non-complete and connected graph, the value $\tilde{\kappa}(\mathcal{G}[U], \boldsymbol{c}_U)$ can be computed in polynomial time as follows. We consider all pairs of nonadjacent vertices $u, v \in U$ and solve a minimum u-v cut problem on an auxiliary directed network. The value $\tilde{\kappa}(\mathcal{G}[U], \boldsymbol{c}_U)$ is then equal to the minimum u-v cut value obtained over all these pairs. In the auxiliary directed network, vertices u and v are the source and the sink, respectively, whereas each other vertex w is split into two nodes w_1 (the "in" copy) and w_2 (the "out" copy), connected by an arc (w_1, w_2) with capacity c_w. For each edge $\{p, q\}$ of the original graph, we add infinite-capacity arcs from the "out" copy of one endpoint to the "in" copy of the other endpoint. In particular, if both endpoints are split, we add arcs (p_2, q_1) and (q_2, p_1). Otherwise, if one endpoint, say p, coincides with u or v, then the added arcs are (p, q_1) and (q_2, p), since u and v are not split. Any finite-capacity minimum u-v cut in this auxiliary network therefore cuts only arcs of the form (w_1, w_2). This construction follows the standard way of computing the vertex connectivity of a graph, with the only difference that, in our case, we set the capacities of the arcs (w_1, w_2) to the weights of the corresponding vertices w, instead of 1. As an illustration, consider the example graph $\mathcal{G}[U]$ displayed in Figure 6a, where $U = \{u, p, q, v\}$ and $\mathcal{E}[U] = \{\{u, p\}, \{p, q\}, \{q, v\}\}$. To illustrate the construction of the auxiliary network, we

consider the pair of nonadjacent vertices $\{u, v\}$.Hence, vertex p is split into p_1 and p_2, connected by an arc of capacity c_p. Similarly, vertex q is split into q_1 and q_2, connected by an arc of capacity c_q. The source node u and the sink node v are linked with infinite-capacity arcs to their neighbors (see Figure 6b). Assuming $c_p < c_q$, a minimum u-v cut is crossed by arc (p_1, p_2) and has value c_p.

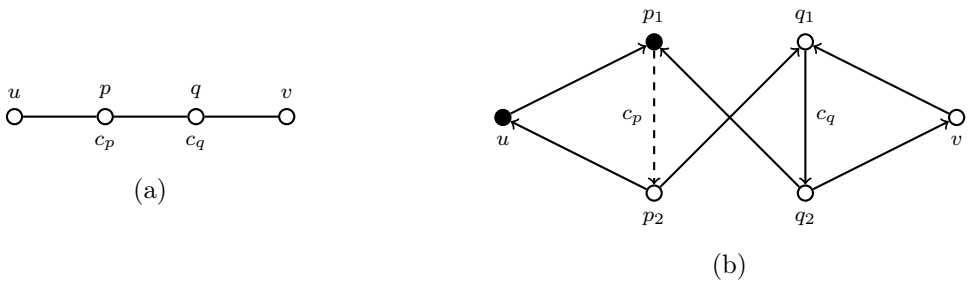


Figure 6: Auxiliary network construction for a graph $\mathcal{G}[U]$, where $U = \{u, p, q, v\}$ and $\mathcal{E}[U] = \{\{u, p\}, \{p, q\}, \{q, v\}\}$. In the right subfigure, the dashed arc (p_1, p_2) belongs to a minimum u-v cut of value c_p. Black-filled nodes belong to the source side of the cut, while white nodes belong to the sink side. Arcs without labels have infinite capacity.

Preliminary computational experiments indicate that the effectiveness of the connectivity cut depends on the value of k. For moderate values (up to $k = 15$), the cut typically strengthens the LP relaxation and improves the overall performance, while incurring only a limited computational overhead. For larger values of k, the cut may instead become less beneficial and interfere with the internal branching mechanisms of the solver. For this reason, in our computational study we activate the connectivity cut only for $k \leq 15$.

D The Iterative Disconnection Heuristic

Our heuristic, denoted by the Iterative Disconnection Heuristic (IDH), aims to find a set $\mathcal{V}_0 \subseteq \mathcal{V}$ such that $\mathcal{G}[\mathcal{V} \setminus \mathcal{V}_0]$ has at least k connected components, i.e., a heuristic solution to the k-VCP. Starting with an empty set $\mathcal{V}_0 \leftarrow \emptyset$, IDH repeatedly considers the induced subgraph $\mathcal{G}[\mathcal{V} \setminus \mathcal{V}_0]$. As long as this subgraph has fewer than k connected components, the heuristic proceeds as follows. For each non-complete connected component $\mathcal{G}[H]$ of $\mathcal{G}[\mathcal{V} \setminus \mathcal{V}_0]$, with $H \subseteq \mathcal{V} \setminus \mathcal{V}_0$, let $\boldsymbol{c}_H = (c_v)_{v \in H}$ and compute $\tilde{\kappa}(\mathcal{G}[H], \boldsymbol{c}_H)$, together with a minimum-cost vertex set $W_H \subseteq H$ whose removal disconnects $\mathcal{G}[H]$ (see Appendix C for the definition of the notation). IDH then selects a component $H^\star$ attaining the smallest value of $\tilde{\kappa}(\mathcal{G}[H], \boldsymbol{c}_H)$ and updates $\mathcal{V}_0 \leftarrow \mathcal{V}_0 \cup W_{H^\star}$. If, after this first iteration, $\mathcal{G}[\mathcal{V} \setminus \mathcal{V}_0]$ already has at least k connected components, then the solution found is optimal for the k-VCP: indeed, any feasible solution must remove vertices of total cost at least $\tilde{\kappa}(\mathcal{G}, \boldsymbol{c})$, and IDH has removed a vertex set attaining this bound. The heuristic terminates once $\mathcal{G}[\mathcal{V} \setminus \mathcal{V}_0]$ has at least k connected components. If, before reaching this condition, all connected components of $\mathcal{G}[\mathcal{V} \setminus \mathcal{V}_0]$ are complete graphs, IDH fails, since no vertex removal can disconnect any of the remaining components. In this case, the heuristic does not find a solution, even if one may exist.

E Benchmark instances

For our numerical experiments, we use the same instances as Furini et al. (2020). Their test set consists of 118 graphs, where 50 originate from the 2nd DIMACS challenge (called the `Coloring` family), 9 come from the 10th DIMACS challenge on graph partitioning and clustering (called the `Partitioning` family), and 59 intersection graphs from de Souza & Balas (2005) (called the `Intersection` family) which have been proposed for the vertex separator problem. Following Furini et al. (2020), we consider the k-VCP for $k \in \{5, 10, 15, 20\}$ for these graphs.

Across these three families of graphs, the number of vertices ranges from 11 to 297 (with an average of about 101), and the number of edges ranges from 20 to 7763 (with an average of about 1426). On average, instances in the `Coloring` and `Intersection` families tend to have substantially more edges (typically between one and several thousand), whereas the `Partitioning` family contains graphs with noticeably larger vertex sets (on average close to 150 vertices). Tables G.2–G.5 provide for each graph in our test set the exact number of vertices and edges as well as the stability number. Recall that the stability number $\alpha(\mathcal{G})$ of a graph $\mathcal{G}$ is the maximum size of a stable set in $\mathcal{G}$.

Due to Lemma 10, an instance of the k-VCP for graph $\mathcal{G}$ is feasible if and only if $\alpha(\mathcal{G}) \geq k$. As Furini et al. (2020), we therefore consider, for fixed $k \in \{5, 10, 15, 20\}$, only graphs $\mathcal{G}$ with $\alpha(\mathcal{G}) \geq k$. For $k = 5$, all graphs in our test set have a stability number of at least 5, i.e., no graph is discarded. Following Furini et al. (2020), we further simplify an instance $(\mathcal{G}, k)$ of the k-VCP by (i) identifying vertices that must belong to any feasible k-vertex cut for the given k; such vertices must be fixed to the vertex cut and removed from the graph; and (ii) instances whose reduced graphs already contain at least k connected components are eliminated because they are trivial. For details on this process, we refer to Furini et al. (2020), which also reports the number of vertices and edges of the graphs after preprocessing. This reduction is effective for a substantial subset of instances: depending on the value of k, between 11 and 20 graphs undergo vertex deletions, and while many reductions remove only a few vertices, in some cases more than one hundred vertices are eliminated, leading in several instances to a substantial shrinkage of the input graph.

After the filtering and reduction procedure, a total of 304 feasible and non-trivial k-VCP instances are obtained. Table E.1 reports the number of instances for each family of graphs and each value of k. Based on these graphs we define two test sets. The *unweighted* test set consists of all 304 graphs with unit vertex costs. The *weighted* test set uses the same graphs, but assign each vertex $v \in \mathcal{V}$ a cost c_v drawn uniformly from $\{1, 2, \ldots, 10\}$.

The preprocessing and filtering of instances as described above is implemented in `Gurobi` 11.0.3 and is applied off-line to all instances.

F Separation of the clique-partitioning constraints

The clique-partitioning constraints (4d) of ILP_{E1} and ILP_{E2} guarantee that the subsets of vertices selected by the variables $\boldsymbol{\lambda}$ are pairwise disjoint on the induced graph $\mathcal{G}[\mathcal{V} \setminus \mathcal{V}_0]$. Since the family of clique-partitioning constraints can be exponentially large, it is practically infeasible to explicitly add all of them to (4) though. As observed by Cornaz et al. (2019), to obtain a

Table E.1: Number of k-VCP instances used in the experiments, grouped by family of graphs and value of k.

Family	$k = 5$	$k = 10$	$k = 15$	$k = 20$	Total
`Coloring`	42	32	29	28	131
`Partitioning`	9	9	9	8	35
`Intersection`	56	39	27	16	138
Total	107	80	65	52	304

valid formulation, it is sufficient to consider a family of clique-partitioning inequalities associated with a clique cover of the edge set. This is the approach adopted in formulation ILP_{E1}, where a collection of maximal cliques is used to partition all edges. The quality of the LP relaxation, however, strongly depends on the family of clique-partitioning constraints that is selected, cf. Section 4.5.

A natural question is to understand which dual bound can be obtained when we explicitly separate violated clique-partitioning inequalities, instead of restricting ourselves to those associated with a fixed family of cliques. To address this question, we introduce and analyze the separation problem for the clique-partitioning inequalities. Given a vector $\boldsymbol{\lambda} \in \mathbb{R}^{\mathscr{S}}$, the separation problem of clique-partitioning inequalities is to decide whether there exists a clique $C \in \mathscr{C}$ such that $\sum_{S \in \mathscr{S}(C)} \lambda_S > 1$. In the following, we prove that the separation problem is $\mathcal{NP}$-hard for solutions of a restricted master problem.

Proposition 13. *The separation problem for clique-partitioning inequalities in $\mathcal{NP}$-hard.*

Proof. Proof We prove the claim by a reduction from the $\mathcal{NP}$-hard Maximum Clique Problem (MCP). That is, given an undirected graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ and a positive integer $\ell \geq 2$, one needs to decide whether there exists a clique of size at least ℓ in $\mathcal{G}$. We construct an instance of the separation problem as follows.

Let $\mathscr{S}$ be the family of all singleton subsets of $\mathcal{V}$, i.e., $\mathscr{S} = \{\{v\} : v \in \mathcal{V}\}$. For each $S = \{v\} \in \mathscr{S}$, set $\lambda_S = \frac{1}{\ell - 1}$. On the one hand, if there exists a clique C of size at least ℓ in $\mathcal{G}$, then $\sum_{S \in \mathscr{S}(C)} \lambda_S = \sum_{v \in C} \lambda_{\{v\}} \geq \frac{\ell}{\ell - 1} > 1$, i.e., there exists a violated clique-covering inequality. On the other hand, if there exists a violated clique-covering inequality, there exists a clique C in $\mathcal{G}$ with $\sum_{v \in C} \lambda_{\{v\}} = \sum_{S \in \mathscr{S}(C)} \lambda_S > 1$. Since all $\boldsymbol{\lambda}$-variables have the same value $\frac{1}{\ell - 1}$, this means that the left-hand side of this inequality evaluates to $\frac{|C|}{\ell - 1} > 1$. That is, C has size at least ℓ, which concludes the proof. □ □

Due to the separation problem being $\mathcal{NP}$-hard, we cannot expect to find a polynomial time algorithm for separating clique-covering inequalities. We therefore suggest to solve the separation problem by means of integer programming. To this end, observe that there is a clique-covering inequality that is violated by a non-negative vector $\boldsymbol{\lambda}$ if and only if

$$\max_{C \in \mathscr{C}} \sum_{S \in \mathscr{S}(C)} \lambda_C > 1.$$

We can find a maximizing clique C by solving an integer program. Let us define a variable $\beta_v \in \{0, 1\}$, for all $v \in \mathcal{V}$, taking value 1 if vertex v is in C, and 0 otherwise; we also define a

variable $\gamma_S \in \{0,1\}$ for all $S \in \mathscr{S}$, equal to 1 if clique C intersects S, and 0 otherwise. An ILP formulation for the separation problem reads as follows:

$$\text{(SEP)} \qquad \max_{\substack{\boldsymbol{\beta}\in\{0,1\}^{\mathcal{V}} \\ \boldsymbol{\gamma}\in\{0,1\}^{\mathscr{S}}}} \sum_{S\in\mathscr{S}} \lambda_S \gamma_S \tag{14a}$$

$$\beta_u + \beta_v \leq 1, \qquad \{u,v\} \in \overline{\mathcal{E}}, \tag{14b}$$

$$\gamma_S \leq \sum_{v\in S} \beta_v, \qquad S \in \mathscr{S}. \tag{14c}$$

where $\overline{\mathcal{E}} = \{\{u,v\} : u,v \in \mathcal{V} \text{ and } \{u,v\} \notin \mathcal{E}\}$ is the set of non-edges of $\mathcal{G}$. The objective function (14a) maximizes the total weight of the subsets in $\mathscr{S}$ that are intersected by the selected clique, where the weight of each subset S is given by the value λ_S of the corresponding variable in the current fractional solution. Constraints (14b) ensure that, for each non-edge $\{u,v\} \in \overline{\mathcal{E}}$, at most one of its endpoints can be part of the selected clique, thus guaranteeing that any subset of vertices $C = \{v \in V : \beta_v = 1\}$ is a clique. Finally, Constraints (14c) enforce that, for each subset of vertices $S \in \mathscr{S}$, its associated variable γ_S can take value 1 only if at least one of the vertices in S is part of the solution clique C. We remark that, since $\lambda_S \geq 0$ for all $S \in \mathscr{S}$, any solution $(\boldsymbol{\beta}, \boldsymbol{\gamma})$ to (14) with $\gamma_S < \min\{1, \sum_{v\in S} \beta_v\}$ for some $S \in \mathcal{S}$ cannot be optimal. This is because setting $\gamma_S = \min\{1, \sum_{v\in S} \beta_v\}$ we obtain a solution with a greater objective function value, without violating any constraint. It follows that the integrality of the $\boldsymbol{\beta}$ variables suffices to guarantee that the $\boldsymbol{\gamma}$ variables are integral. As a consequence, the integrality of the latter set of variables can be relaxed.

We now turn to a computational assessment of the effectiveness of the exact separation procedure described above. Given the $\mathcal{NP}$-hardness result of Proposition 13, we restrict the use of Model (14) to the root node of the branch-and-bound tree. In this way, we can evaluate the impact of optimizing over the closure of the clique-partitioning constraints at the root node, without incurring an excessive computational overhead. In our experiments, we compare two settings on the entire set of unweighted instances. In the first one, denoted by BP*, we do not perform any separation of clique-partitioning inequalities. Instead, we generate at the beginning a family of maximal cliques that covers all edges of the graph, and we add the corresponding clique-partitioning constraints to the formulation. In the second setting, denoted by BP*-C, we start from the same initial family of maximal cliques and then apply the exact separation procedure at the root node, by solving (14) iteratively until no violated clique-partitioning inequality is found. The initial family of maximal cliques used in both settings is constructed by means of the greedy edge-cover procedure detailed in Section 3.1.1. The resulting family of maximal cliques provides an edge cover of $\mathcal{E}$ and is used to define the initial set of clique-partitioning constraints in both BP* and BP*-C.

Table F.1 reports a comparison between the BP*-C and BP* settings at the root node. The table includes only the instances for which BP*-C generates at least one violated clique-partitioning inequality at the root node. Over these 47 instances, BP*-C generates on average 8.5 clique-partitioning cuts at the root node, with a maximum of 44 cuts on a single instance. Rows are grouped by the value of parameter k. For each value of k, column "# inst" reports

Table F.1: Evaluation of the effect of separating clique-partitionig inequalities at the root node. The last row reports the total number of instances for the "# inst" column and average values for the remaining columns.

		BP⋆-C				BP⋆			
		LP time		integrality gap		LP time		integrality gap	
k	# inst	avg	time	avg	max	avg	max	avg	max
5	22 / 107	9.7	35.3	30.7	63.5	0.8	4.6	32.6	63.6
10	14 / 80	18.5	90.3	17.7	59.0	1.2	5.1	19.7	59.0
15	7 / 65	30.9	113.6	22.7	55.3	0.8	1.2	24.0	55.4
20	4 / 52	16.9	31.2	18.9	49.5	0.7	0.8	20.6	49.5
Overall	47 / 304	16.1		24.6		0.9		26.4	

the number of instances for which BP⋆-C generates at least one violated clique-partitioning inequality at the root node, over the total number of instances with that value of k. The first part of Table F.1 (columns *integrality gap*) reports the integrality gap, defined as the percentage difference between the best known integer solution value and the optimal value of the LP relaxation at the root node for a given setting, taken with respect to the best known integer solution value, i.e, $100 \cdot \frac{z^{\text{BEST}} - z^{\text{LP}}}{z^{\text{BEST}}}$, where z^{BEST} is the best known integer solution objective value and z^{LP} is the optimal value of the root-node LP relaxation under the considered setting. For each k, we then report the average and maximum gap over the corresponding instances. Columns "LP time" report instead the average and maximum CPU time (in seconds) to solve the root-node LP relaxation in each setting.

We observe that the use of exact separation at the root node significantly increases the LP time. For example, for $k = 5$ the average root-node time grows from 0.8 seconds in BP⋆ to 9.7 seconds in BP⋆-C and for $k = 10$ it grows from 1.2 to 18.5 seconds. On average over all values of k, the root-node time increases from 0.9 seconds for BP⋆ to 16.1 seconds for BP⋆-C. Similar trends can be seen for the maximum times, where BP⋆-C frequently requires one to two orders of magnitude more CPU time than BP⋆ to process the root node. In contrast, the improvement in the root-node integrality gap obtained by BP⋆-C is limited. For $k = 5$, the average gap decreases only from 32.6% to 30.7%, and for $k = 10$ from 19.7% to 17.7%. For larger values of k, the average gaps of the two settings are very close. Overall, the average gap over all instances with cuts decreases from 26.4% in BP⋆ to 24.6% in BP⋆-C, which corresponds to a rather modest improvement compared with the increase in root-node time. We also evaluated the impact of exact separation at the root node on the overall performance of the algorithm. Running the full algorithm with BP⋆-C (separating clique-partitioning inequalities only at the root node) does not lead to any noticeable advantage compared with BP⋆ in terms of total solution time or number of explored nodes. On the contrary, when looking at the optimality gaps at the time limit on the subset of instances that remain unsolved, the behavior of BP⋆-C is slightly worse. On the 28 out of 47 instances with root-node cuts that are not solved to proven optimality within the time limit, the average optimality gap –computed as $100 \cdot (\text{UB} - \text{LB})/\text{UB}$ at the time limit– is 29.08%

for BP*-C and 28.88% for BP*. These results indicate that, for the instances considered, the exact separation of clique-partitioning inequalities at the root node is not an effective strategy. The reduction in the root-node integrality gap is marginal and does not translate into better global performance, while the additional cost in terms of LP time is significant.

G Supplementary detailed computational results

Figure 7 reports performance profiles for COMP, HYB, and BP* on the unweighted (left column) and weighted (right column) instances, for each value of k. Overall, the plots confirm the already observed good behavior of BP*, but they also highlight some regimes where COMP or HYB can be competitive. For unweighted instances, BP* is generally the most robust algorithm: its profile is typically above that of HYB, which in turn dominates COMP for most values of the performance ratio τ and for most k. In particular, for $k \in \{10, 15, 20\}$ the curve of BP* remains clearly above that of COMP over the whole range of τ, while the gap with HYB depends on k and tends to decrease for larger values of k. The performance difference is less pronounced on weighted instances. For $k = 5$, COMP has the best profile, especially for large values of τ, while BP* dominates HYB over the entire range. For $k = 10$, BP* keeps dominating HYB for all values of τ, while for $k = 15$ and $k = 20$ the two algorithms have very close profiles, although the curve of BP* eventually overtakes that of HYB. The performance of COMP deteriorates as the value of k increases, and its profile stays well below both HYB and BP* for all $k \in \{10, 15, 20\}$. Taken together, these profiles indicate that BP* is overall the most effective algorithm, particularly on unweighted instances and moderate values of k, while COMP can still be preferable on the easiest instances with small k (particularly if weighted), and HYB remains competitive on all the instances, but mostly on weighted instances with higher values of k.

Table G.1 reports a detailed performance comparison of algorithms COMP, HYB and BP* on the full benchmark set, separately for unweighted and weighted instances. For each family of instances and each value of k, the table lists the number of instances considered (#inst) and, for each algorithm, the number of instances solved to proven optimality (#opt), the average running time in seconds on solved instances (time), the average optimality gap at the time limit, computed over unsolved instances only (gap), and the average number of explored branch-and-bound nodes (nodes) on solved instances. To quantify how far an algorithm is from proving optimality when the time limit expires, we measure an *optimality gap* based on its best primal and dual information: let z^{UB} denote the best incumbent value found by an algorithm at time limit and z^{LB} the best dual bound available at that time. For each instance that is not closed to proven optimality within the time limit, we define the optimality gap as $100 \cdot \frac{z^{\mathrm{UB}} - z^{\mathrm{LB}}}{z^{\mathrm{UB}}}$, which is therefore expressed as a percentage. If an algorithm fails to produce a valid primal or dual bound before the time limit, we conservatively set the optimality gap to 100%, corresponding to the worst possible case.

Beyond the number of instances solved, already discussed in Section 4.4, BP* also tends to explore substantially fewer branch-and-bound nodes on the instances that it solves. This is consistent with the tighter LP relaxation bound provided by the ILP$^\star_{\mathrm{E2}}$ formulation, which leads to stronger pruning in the search tree. In many families and values of k, BP* also has

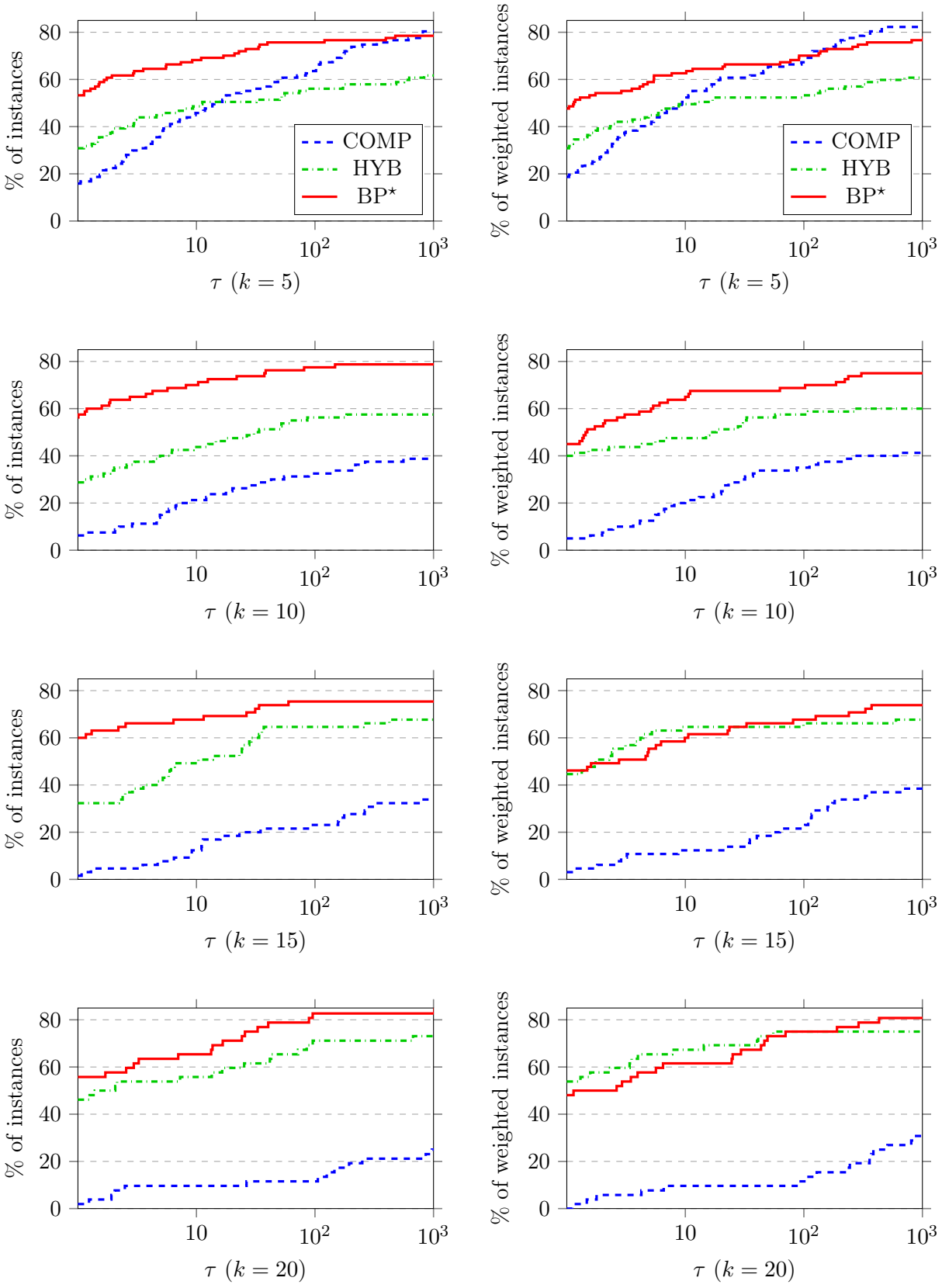


Figure 7: Performance profiles of algorithms COMP, HYB and BP⋆ on the full set of unweighted and weighted benchmark instances, with a time limit of one hour.

smaller average optimality gaps on the unsolved instances. These features are visible in both the unweighted and weighted sections of the table. The distribution of the results across families reflects the complementary strengths of the algorithms. For `Intersection` instances, BP* clearly dominates: it solves almost all unweighted and weighted cases, often with fewer nodes and smaller gaps than HYB and COMP. For Partitioning instances, the picture is more mixed, with BP*, HYB, and COMP each taking the lead in different (family, k) combinations, but BP* still maintaining competitive performance in terms of number of solved instances. Finally, on `Coloring` instances, BP* remains competitive but less dominant: HYB and COMP solve more instances in several k configurations, often closing them in a smaller amount of time as well.

In Tables G.2-G.5 we report, for each instance family, the best known solution values for all values of k for which the instances are defined. For each instance, the tables list the number of vertices $|\mathcal{V}|$, the number of edges $|\mathcal{E}|$, the stability number $\alpha(G)$, and the best known objective value for each considered value of $k \in \{5, 10, 15, 20\}$. These values are either proven optimal or represent the best solutions found in the literature or by our algorithm BP*. Throughout these tables, bold values indicate proven optimal solution values. Values annotated with a red star ($\star$) are new optimal solution values obtained by our algorithm BP* within a time limit of one hour on our machines. Values marked with a dagger superscript ($\dagger$) are optimal solution values that are attained by at least one among the algorithms HYB or COMP, but not by BP* in our experiments. Finally, values with a double dagger superscript ($\ddagger$) are optimal solution values already known in the literature, but not matched by any of the algorithms considered in our computational study. Finally, instances denoted by a dot (.) are trivial instances solved in preprocessing. For unweighted instances only, we also performed an additional campaign of long runs with a time limit of three days. These runs were restricted to a subset of instances that were not solved to optimality by any algorithm in the literature, including those tested in Furini et al. (2020). The goal of this experiment was to assess how many of these open instances could be closed when allowing for a substantially larger computational effort. The instances that we were able to close in this setting are annotated with the red double star ($\star\star$) in the tables.

The comparison with Furini et al. (2020) must be interpreted with care, because the computational results therein were obtained on more powerful machines than those available for our experiments. As a consequence, HYB and COMP solve a larger number of instances in Furini et al. (2020) than what is reflected by our own runs of these algorithms. All counts reported henceforth, including the classification into closed and open instances, are therefore based both on executions carried out on our hardware and on results from the literature. Overall, 19 unweighted instances and 22 weighted instances are still open. BP* closes 33 unweighted instances that were previously open in the literature when using a time limit of one hour on our machines, and closes 6 additional unweighted instances when the time limit is extended to three days. For weighted instances, BP* closes 40 instances that were previously open in the literature. Most of these new optimal solutions concern the `Intersection` family: 30 unweighted and 32 weighted instances from this family are closed for the first time by BP*. On the other hand, there remain several instances that BP* does not solve to optimality within the imposed time limits, while other algorithms do. For unweighted instances, there are 37 such cases, either solved by HYB or COMP in Furini et al. (2020) or by our own runs of these algorithms, but not by BP*.

Table G.1: Performance comparison of algorithms COMP, HYB and BP* on the full set of benchmark instances, with a time limit of one hour.

			BP*				**HYB**				**COMP**			
Family	k	#inst	#opt	time	gap	nodes	#opt	time	gap	nodes	#opt	time	gap	nodes
Unweighted Instances														
`Coloring`	5	42	23	54.8	42.3	20.3	26	131.0	70.6	45.0	**32**	250.8	61.9	19,597.8
	10	32	20	222.5	35.6	8.8	**22**	65.4	52.0	15.9	11	358.6	63.2	30,866.1
	15	29	19	71.5	41.4	3.9	**25**	78.3	68.5	14.1	11	302.1	64.1	30,689.5
	20	28	21	172.3	37.4	39.9	**26**	125.1	61.7	22.4	10	874.9	68.0	91,338.9
`Partitioning`	5	9	**8**	273.6	23.2	5.6	**8**	40.2	72.8	11.0	**8**	27.0	29.3	2,627.4
	10	9	5	19.6	59.0	10.2	**8**	228.7	83.5	12.1	7	274.7	60.8	2,414.6
	15	9	6	39.9	70.6	4.0	**7**	37.4	83.3	6.0	6	261.5	56.2	15,589.7
	20	8	**7**	33.6	100.0	12.4	6	225.8	61.0	19.3	2	9.7	44.9	1,176.0
`Intersection`	5	56	**54**	91.4	57.9	78.9	33	207.3	54.9	145.0	48	201.2	54.0	38,283.7
	10	39	**38**	146.2	15.7	113.8	16	208.4	51.4	507.2	14	18.0	62.2	5,716.1
	15	27	**25**	34.0	18.4	16.4	12	97.6	50.0	61.1	11	133.5	70.2	14,642.1
	20	16	**16**	28.5	-	25.8	6	18.6	48.4	63.5	5	10.7	65.0	1,091.2
Tot		304	**242**				195				165			
Weighted Instances														
`Coloring`	5	42	24	336.3	45.1	29.4	25	78.3	62.2	42.3	**33**	287.0	66.2	20,756.3
	10	32	18	51.0	42.0	17.3	**22**	115.7	50.7	31.8	14	215.3	59.2	29,885.9
	15	29	20	368.3	52.6	44.0	**25**	191.9	61.0	24.3	13	671.4	62.0	62,914.5
	20	28	20	331.7	40.0	38.4	**26**	228.1	62.9	36.0	10	712.6	53.2	17,182.6
`Partitioning`	5	9	7	35.9	100.0	38.0	8	14.7	69.6	2.4	**9**	376.4	-	33,411.7
	10	9	7	510.1	60.5	9.1	**8**	178.2	81.5	16.9	6	34.1	38.2	1,291.7
	15	9	6	232.9	69.6	11.7	**7**	43.3	79.3	13.6	5	216.3	43.1	21,711.4
	20	8	**7**	316.8	100.0	18.6	6	262.1	58.0	21.5	3	865.2	47.8	116,003.7
`Intersection`	5	56	**54**	119.7	59.2	30.4	35	210.5	53.1	160.1	48	185.0	39.6	25,108.7
	10	39	**36**	26.3	43.0	25.3	19	230.4	51.2	206.6	14	26.5	57.2	8,957.6
	15	27	**24**	57.1	14.4	9.2	13	35.7	40.9	84.2	10	11.3	60.2	5,592.8
	20	16	**16**	77.6	-	24.4	7	73.0	44.4	45.3	5	457.2	59.5	92,402.4
Tot		304	**239**				201				170			

For weighted instances, this situation occurs 43 times. The majority of these missed instances

belong to the `Coloring` family (30 in the unweighted case and 33 in the weighted case), which indicates that the approach underlying BP* has more difficulty with `Coloring` instances, while being particularly effective on `Intersection` instances. This behavior is in contrast with HYB and COMP, which tend to perform better on `Coloring` and worse on `Intersection`.

Table G.2: Best known values for unweighted instances from the `Coloring` and `Partitioning` families.

				k								k			
Instance	$\|\mathcal{V}\|$	$\|\mathcal{E}\|$	$\alpha(G)$	5	10	15	20	Instance	$\|\mathcal{V}\|$	$\|\mathcal{E}\|$	$\alpha(G)$	5	10	15	20
1-FullIns_3	30	100	14	**7**	**11**			mug88_1	88	146	29	**4**	**9**	**15**	**20**
1-FullIns_4	93	593	45	**9**†	**13**	**18**†	**22**	mug88_25	88	146	29	**4**	**9**	**14**	**19**
1-Insertions_4	67	232	32	**7**	**12**	**16**	**22**	mulsol.i.2	188	3885	90	.	.	.	**18**
2-FullIns_3	52	201	25	**8**	**13**	**17**	**23**	mulsol.i.3	184	3916	86	.	.	**18**	**19**
2-Insertions_3	37	72	18	**6**	**10**	**16**		mulsol.i.4	185	3946	86	.	.	**18**	**19**
2-Insertions_4	149	541	74	**7**†	**11**	**17**‡	**22**†	mulsol.i.5	186	3973	88	.	.	**18**	**19**
3-FullIns_3	80	346	37	**9**†	**14**†	**17**	**25**	myciel3	11	20	5	.			
3-Insertions_3	56	110	27	**6**	**11**	**16**	**21**	myciel4	23	71	11	**7**	**12**		
4-FullIns_3	114	541	55	**9**†	**15**†	**18**†	**23**†	myciel5	47	236	23	**8**	**13**	**18**	**23**
4-Insertions_3	79	156	39	**6**	**11**	**16**	**21**	myciel6	95	755	47	**9**†	**14**†	**19**†	**24**†
5-FullIns_3	154	792	72	**9**†	**15**‡	**19**†	**22**†	myciel7	191	2360	95	**10**†	**15**‡	**20**†	**25**†
anna	138	493	80	**1**	**1**†	**2**†	**2**†	queen10_10	100	1470	10	70	**90**		
david	87	406	36	.	.	**4**	**9**	queen11_11	121	1980	11	82	108		
DSJC125.1	125	736	34	21	42	60	72	queen12_12	144	2596	12	92	130		
DSJC125.5	125	3891	10	102	**115**			queen13_13	169	3328	13	105	151		
games120	120	638	22	19	**39****	**55****	**67**	queen14_14	196	4186	14	117	174		
huck	74	301	27	**1**	**3**	**6**	**9**	queen5_5	25	160	5	**20**			
jean	80	254	38	**1**	**1**	**2**	**4**	queen6_6	36	290	6	**28**			
miles1000	128	3216	8	**53**				queen7_7	49	476	7	**38**†			
miles1500	128	5198	5	**115**				queen8_12	96	1368	8	67			
miles250	128	387	44	.	.	**4**†	**11**	queen8_8	64	728	8	**48**†			
miles500	128	1170	18	**7**	**25***	**55***		queen9_9	81	1056	9	**59**‡			
miles750	128	2113	12	**20**	**75**			r125.1	125	209	49	.	.	**1**	**5**
mug100_1	100	166	33	**5**	**10**	**15**	**20**	r125.1c	125	7501	7	**116**			
mug100_25	100	166	33	**5**	**10**	**15**	**20**	r125.5	125	3838	5	**91**			
adjnoun	112	425	53	**2**	**6**†	**11**	**16**	jazz	198	2742	40	**4**	**12**†	**25**‡	**44***
celegansneural	297	2148	110	**1**	**1**†	**2**†	**6**†	karate	34	78	20	**2**	**4**	**6**	**11**
chesapeake	39	170	17	**7**	**12**	**17**		lesmis	77	254	35	**1**	**2**	**3**	**5**
dolphins	62	159	28	**2**	**7**	**13**	**19**	polbooks	105	441	43	**8**	**15**	**19**	**25**
football	115	613	21	**21****	**43****	**60****	**71**								

Table G.3: Best known values for unweighted instances from the `Intersection` family.

				k								k			
Instance	$\|\mathcal{V}\|$	$\|\mathcal{E}\|$	$\alpha(G)$	5	10	15	20	Instance	$\|\mathcal{V}\|$	$\|\mathcal{E}\|$	$\alpha(G)$	5	10	15	20
arc130	130	7763	6	**83**				L120.fidap022	120	4307	5	**87**			
ash219	85	219	29	**7**	**16**	**26**	**34**	L120.fidap025	120	2787	5	.			
ash331	104	331	30	**8**	**21**	**33***	**45***	L120.fidapm02	120	4626	5	**91**			
ash85	85	616	14	**22**	**46***			L120.rbs480a	120	3273	6	**76**			
bcspwr01	39	118	13	**7**	**16**			L120.wm2	120	3387	23	**3**†	**8**	**13**	**41**
bcspwr02	49	177	16	**7**	**16**	**24**		L125.ash608	125	390	37	**8**	**19***	**30***	**41***
bcspwr03	118	576	32	**10**	**23**	**35**	**46**	L125.bcsstk05	125	2701	9	**41**			
bfw62a	62	639	8	**22**				L125.can__161	125	1257	15	**36***	**74***	**102***	
can61	61	866	6	**39**				L125.can__187	125	1022	20	**26***	**54***	**76***	**102**
can62	62	210	18	**7**	**17**	**27**		L125.dwt__162	125	943	16	**23***	**50***	**80***	
can73	73	652	13	**28**	**52***			L125.dwt__193	125	2982	8	**56**			
can96	96	912	10	**38***	**72***			L125.fs_183_1	125	3392	9	**16**			
can__144	144	1656	12	**45***	**90***			L125.gre__185	125	1177	19	**27**	60	89	
curtis54	54	337	9	**16**				L125.lop163	125	1218	17	**26****	**54***	97	
dwt66	66	255	13	**15**	**35***			L125.west0167	125	444	39	**5**	**11**	**17**	**24**
dwt72	72	170	24	**7**	**16**	**26**	**36**	L125.will199	125	386	45	**5**	**13**	**20**	**27**
dwt87	87	726	16	**11**	**29**	**54**		L80.cavity01	80	1201	31	**10**	**10**	**20**	**31**
dwt___59	59	256	15	**10**	**25**	**41**		L80.fidap025	80	1201	5	.			
gre__115	115	576	33	**12**	**24**	**38***	**51***	L80.steam2	80	1272	6	**48**			
ibm32	32	179	8	**16**				L80.wm1	80	1786	15	**15**	**36**	**49**	
impcol_b	59	329	20	**5**	**13**	**23**	**38**	L80.wm2	80	1848	11	**4**	**48**		
L100.cavity01	100	1844	36	**10**	**19**	**21**	**32**	L80.wm3	80	1739	13	**4**	**12**		
L100.fidap025	100	2031	5	.				lund_a	147	2837	10	**55***	**117***		
L100.fidapm02	100	3090	5	**80**				pores_1	30	179	6	**20**			
L100.rbs480a	100	2550	5	**66**				rw136	136	641	39	**7**	**20***	**34***	**48***
L100.steam2	100	1766	6	**56**				steam3	80	712	7	**32**			
L100.wm1	100	2956	17	**15**	**28**	**48**		west0067	67	411	12	**20**	**38***		
L100.wm2	100	3039	12	**4**	**41**			west0132	132	560	39	**5**	**12**	**21**	**29**
L100.wm3	100	2934	15	**4**	**12**	**53**		will57	57	304	10	**7**	**22**		
L120.cavity01	120	2972	36	**10**	**21**	**23**	**32**								

Table G.4: Best known values for weighted instances from the `Coloring` and `Partitioning` families.

				k								k			
Instance	$\|\mathcal{V}\|$	$\|\mathcal{E}\|$	$\alpha(G)$	5	10	15	20	Instance	$\|\mathcal{V}\|$	$\|\mathcal{E}\|$	$\alpha(G)$	5	10	15	20
1-FullIns_3	30	100	14	**35**	**53**			mug88_1	59	256	15	**20**	**43**	**68**	**99**
1-FullIns_4	93	593	45	**35**†	**66**†	**90**†	**122**†	mug88_25	66	255	13	**14**	**38**	**63**	**93**
1-Insertions_4	67	232	32	**40**	**68**	**100**	**125**	mulsol.i.2	72	170	24	.	.	.	**96**
2-FullIns_3	52	201	25	**42**	**71**	**92**	**125**	mulsol.i.3	87	726	16	.	.	**96**	**98**
2-Insertions_3	37	72	18	**18**	**50**	**73**		mulsol.i.4	115	613	21	.	.	**96**	**98**
2-Insertions_4	149	541	74	**42**†	**69**‡	**99**†	**124**†	mulsol.i.5	120	638	22	.	.	**96**	**98**
3-FullIns_3	80	346	37	**33**†	**53**†	**76**†	**106**	myciel3	115	576	33	.			
3-Insertions_3	56	110	27	**22**	**47**	**72**	**95**	myciel4	74	301	27	**38**	**68**		
4-FullIns_3	114	541	55	**40**†	**81**†	**98**†	**127**†	myciel5	32	179	8	**47**	**77**	**105**	**129**
4-Insertions_3	79	156	39	**17**	**43**	**68**	**94**	myciel6	59	329	20	**57**†	**87**†	**115**†	**138**†
5-FullIns_3	154	792	72	**35**†	**72**†	**95**†	**113**†	myciel7	198	2742	40	**67**†	97	125	**148**†
anna	138	493	80	**7**	**7**†	**9**†	**15**†	queen10_10	34	78	20	360	**486**		
david	104	331	30	.	.	**17**	**50**	queen11_11	100	1844	36	413	570		
DSJC125.1	39	118	13	**106**‡	210	291	363	queen12_12	100	2031	5	461	693		
DSJC125.5	49	177	16	560	**645**			queen13_13	100	3090	5	547	833		
games120	62	639	8	**90***	198	**283***	**353***	queen14_14	100	2550	5	639	970		
huck	144	1656	12	**7**	**17**	**33**	**54**	queen5_5	100	1766	6	**103**			
jean	62	210	18	**2**	**4**	**14**	**19**	queen6_6	100	2956	17	**149**			
miles1000	297	2148	110	**297**				queen7_7	100	3039	12	**199**†			
miles1500	39	170	17	**626**				queen8_12	100	2934	15	**339**‡			
miles250	54	337	9	.	.	**7**	**30**	queen8_8	120	2972	36	**239**†			
miles500	87	406	36	**42**	**129***	**272***		queen9_9	120	4307	5	**296**†			
miles750	62	159	28	**120**	**400***			r125.1	120	2787	5	.	.	**2**	**9**
mug100_1	125	736	34	**10**	**27**	**46**	**69**	r125.1c	120	4626	5	**648**			
mug100_25	125	3891	10	**11**	**30**	**52**	**77**	r125.5	120	3273	6	**505**			
adjnoun	112	425	53	**11**†	**29**	**51**	**81**	jazz	61	866	6	**23**†	**70**†	**133**‡	**228***
celegansneural	130	7763	6	**5**	**5**	**15**†	**37**†	karate	73	652	13	**11**	**23**	**34**	**61**
chesapeake	85	219	29	**28**	**60**	**92**		lesmis	96	912	10	**4**	**6**	**13**	**21**
dolphins	85	616	14	**10**	**30**	**66**	**89**	polbooks	80	254	38	**34**	**79**	**103**	**136**
football	118	576	32	**101**	221	314	**384***								

Table G.5: Best known values for weighted instances from the `Intersection` family.

				k								k			
Instance	$\lvert\mathcal{V}\rvert$	$\lvert\mathcal{E}\rvert$	$\alpha(G)$	5	10	15	20	Instance	$\lvert\mathcal{V}\rvert$	$\lvert\mathcal{E}\rvert$	$\alpha(G)$	5	10	15	20
arc130	120	3387	23	**442**				L120.fidap022	184	3916	86	**486**			
ash219	125	390	37	**36**	**78**	**120**	**164**	L120.fidap025	185	3946	86	.			
ash331	125	2701	9	**39**	**101***	**161***	**216***	L120.fidapm02	186	3973	88	**509**			
ash85	125	1257	15	**117**	**240***			L120.rbs480a	11	20	5	**433**			
bcspwr01	125	1022	20	**28**	**70**			L120.wm2	23	71	11	**7**	**28**	**67**	**239**
bcspwr02	125	943	16	**38**	**87**	**133**		L125.ash608	47	236	23	**37**	**90***	**142***	**197***
bcspwr03	125	2982	8	**53**	**113**	**168**	**235**	L125.bcsstk05	95	755	47	**218**			
bfw62a	125	3392	9	**114**				L125.can__161	191	2360	95	**193***	**388***	**543***	
can61	125	1218	17	**207**				L125.can__187	105	441	43	**134***	**270***	**402***	**541**
can62	125	444	39	**31**	**78**	**130**		L125.dwt__162	30	179	6	**109***	**245***	**395***	
can73	125	386	45	**144**	**271***			L125.dwt__193	100	1470	10	**291**			
can96	80	1201	31	**190***	**354***			L125.fs_183_1	121	1980	11	**71**			
can__144	125	1177	19	**217***	**472***			L125.gre__185	144	2596	12	**144***	348	502	
curtis54	80	1201	5	**74**				L125.lop163	169	3328	13	**142***	300	521	
dwt66	80	1786	15	**54**	**155***			L125.west0167	196	4186	14	**19**	**46**	**73**	**109**
dwt72	80	1848	11	**26**	**64**	**105**	**169**	L125.will199	25	160	5	**21**	**60**	**92**	**127**
dwt87	80	1739	13	**66**	**164***	**313**		L80.cavity01	36	290	6	**43**	**49**	**92**	**154**
dwt___59	80	1272	6	**59**	**141**	**226**		L80.fidap025	49	476	7	.			
gre__115	77	254	35	**44**	**108**	**181***	**255***	L80.steam2	96	1368	8	**257**			
ibm32	147	2837	10	**80**				L80.wm1	64	728	8	**88**	**218**	**281**	
impcol_b	128	3216	8	**22**	**58**	**109**	**202**	L80.wm2	81	1056	9	**24**	**264**		
L100.cavity01	128	5198	5	**49**	**91**	**100**	**162**	L80.wm3	125	209	49	**23**	**74**		
L100.fidap025	128	387	44	.				lund_a	125	7501	7	**304***	**661***		
L100.fidapm02	128	1170	18	**443**				pores_1	125	3838	5	**99**			
L100.rbs480a	128	2113	12	**370**				rw136	136	641	39	**40**	**107***	**173***	**249***
L100.steam2	100	166	33	**303**				steam3	80	712	7	**145**			
L100.wm1	100	166	33	**78**	**169**	**274**		west0067	67	411	12	**97**	**188**		
L100.wm2	88	146	29	**20**	**237**			west0132	132	560	39	**21**†	**57**†	**97**‡	**132**
L100.wm3	88	146	29	**20**	**76**	**303**		will57	57	304	10	**33**	**113**		
L120.cavity01	188	3885	90	**49**†	**100**	**115**	**168**								